University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Manual Report	**Dated: 28/06/2026**
Semester: 1st	**Session: 2026-30**
Lab/Project/Assignment #: 1	**CLOs to be covered: CLO**
Lab Title: Input/Output, Data Types, Operators, Conditional Statements (If/Else)	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 1

CLO 1:

Lab Goals/Objectives:

To understand and implement basic data types, input/output operations, and arithmetic operators (addition and division).

To master conditional logic using if, elif, and else structures for decision-making tasks like grading systems.

Equipment Required:

- Hardware: Laptop / PC
- Software: IDE (e.g., PyCharm / VS Code), Python/C Environment.

Theory:

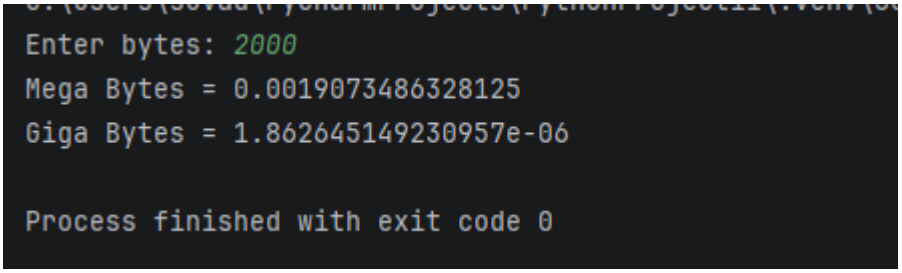
Data Types & I/O: Variables use data types (like integer and float) to store data. Input functions read user data, and output functions display results on the console.
Operators: Arithmetic operators like + (Addition) and / (Division) are used to perform basic mathematical computations.
Conditional Statements: Control structures (if-elif-else) allow the program to make decisions. The program evaluates conditions sequentially, which is ideal for calculating percentages and mapping them to specific grade brackets (e.g., A, B, C).

Lab Work:

Task 1:

```
bytes_value = float(input("Enter bytes: "))

mb = bytes_value / (1024 * 1024)
gb = bytes_value / (1024 * 1024 * 1024)

print("Mega Bytes =", mb)
print("Giga Bytes =", gb)
```

Result:



Task 2:

```
ecat = float(input("Enter ECAT percentage: "))
inter = float(input("Enter Intermediate percentage: "))
matric = float(input("Enter Matric percentage: "))

aggregate = (ecat * 0.33) + (inter * 0.50) + (matric *
0.17)
print("Aggregate =", aggregate)
```

Result:

```
C:\Users\suvad\PycharmProjects\PythonProject11\.venv\Scripts\python.exe C:\Users\su
Enter ECAT percentage: 78
Enter Intermediate percentage:                84
Enter Matric percentage: 78
Aggregate = 81.00000000000001

Process finished with exit code 0
```

Task 3:

```
num = int(input("Enter a number: "))
temp = num
reverse = 0
while temp > 0:
    digit = temp % 10
    reverse = reverse * 10 + digit
    temp = temp // 10
if num == reverse:
    print("Palindrome")
else:
    print("Not Palindrome")
```

Result:

```
C:\Users\suvad\PycharmProjects\PythonProject11\.venv\Scrip
Enter a number: 10
Not Palindrome

Process finished with exit code 0
```

Task 4:

```
import math
x1 = float(input("Enter x1: "))
y1 = float(input("Enter y1: "))
x2 = float(input("Enter x2: "))
y2 = float(input("Enter y2: "))
distance = math.sqrt((x2-x1)**2 + (y2-y1)**2)
print("Distance =", distance)
def quadrant(x, y):
    if x > 0 and y > 0:
        return "First Quadrant"
    elif x < 0 and y > 0:
        return "Second Quadrant"
    elif x < 0 and y < 0:
        return "Third Quadrant"
    elif x > 0 and y < 0:
        return "Fourth Quadrant"
    else:
        return "On Axis"
print("Point 1:", quadrant(x1, y1))
print("Point 2:", quadrant(x2, y2))
```

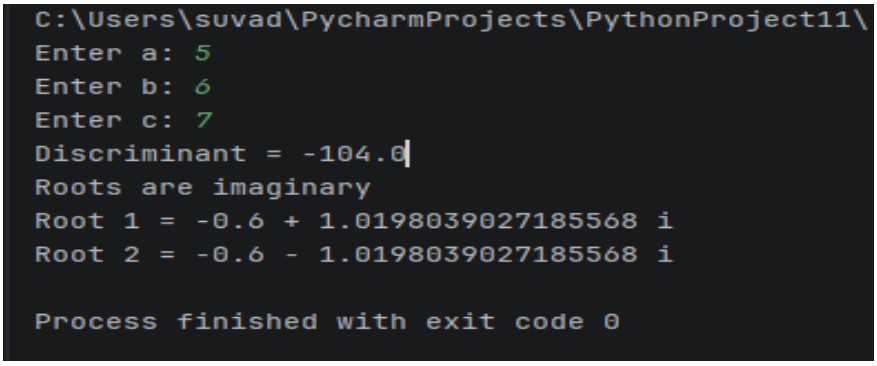
Result:

Task 5:

```
import math

a = float(input("Enter a: "))

b = float(input("Enter b: "))

c = float(input("Enter c: "))

D = b*b - 4*a*c

print("Discriminant =", D)

if D > 0:

    print("Roots are real and distinct")
```

```
elif D == 0:
    print("Roots are real and equal")
else:
    print("Roots are imaginary")
if D >= 0:
    x1 = (-b + math.sqrt(D)) / (2*a)
    x2 = (-b - math.sqrt(D)) / (2*a)
    print("Root 1 =", x1)
    print("Root 2 =", x2)
else:
    real = -b / (2*a)
    imag = math.sqrt(-D) / (2*a)
    print("Root 1 =", real, "+", imag, "i")
    print("Root 2 =", real, "-", imag, "i")
```

Result:



Conclusions:

This lab successfully demonstrated the application of fundamental programming concepts, including data types, arithmetic operators, and user I/O. Through the implementation of if-elif-else structures, a practical understanding of conditional control flow was achieved, effectively enabling dynamic decision-making and grading logic. All tasks were executed with correct outputs.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Manual Report	**Dated: 29/06/2026**
Semester: 1st	**Session: 2026-30**
Lab/Project/Assignment #: 2	**CLOs to be covered: CLO**
Lab Title: For Loops	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 2:

CLO 1:

Lab Goals/Objectives:

- To understand the concept and implementation of iterative control structures using for loops.
- To master nested loop logic by generating different geometric star (*) patterns.

Equipment Required:

- Hardware: Laptop / PC
- Software: IDE (e.g., PyCharm / VS Code), Python/C Environment.

Theory:

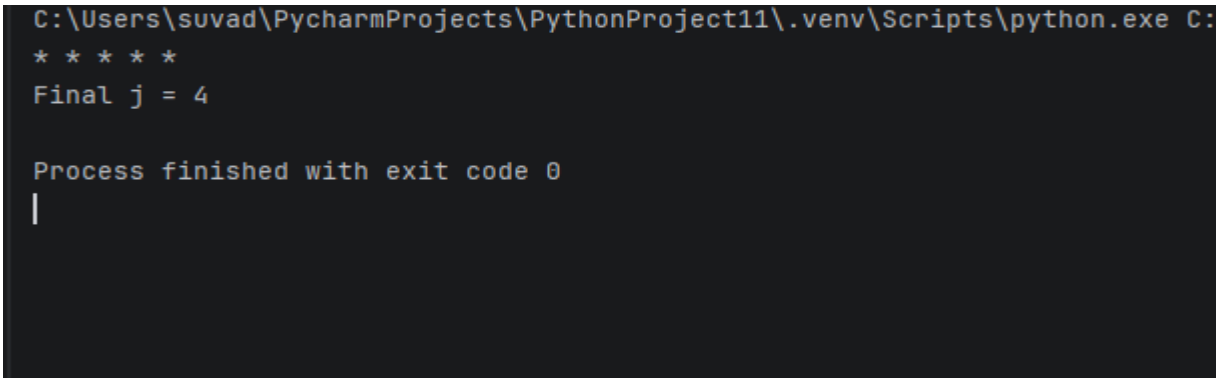
For Loops: A for loop is a control flow statement used to repeat a block of code a specific number of times. It is highly efficient for iterating over ranges or sequences.
Nested Loops: When a loop is placed inside another loop, it is called a nested loop. In pattern printing, the outer loop typically controls the number of rows, while the inner loop manages the number of columns (or stars) printed in each row.

Lab Work:

Task 1:

```
i = 5
for j in range(i):
    print("*", end=" ")
print()
print(f"Final j = {j}")
```

Result:



Task 2:

```
for j in range(1, 10):
    print(j * "*")
```

Result:

Conclusions:

This lab successfully demonstrated the utility of for loops in handling repetitive tasks efficiently. By printing various star patterns, a solid understanding of nested loop logic, loop boundaries, and sequence iteration was achieved. All tasks executed with correct outputs.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Manual Report	**Dated: 29/06/2026**
Semester: 1st	**Session: 2026-30**
Lab/Project/Assignment #: 3	**CLOs to be covered: CLO**

Lab Title: While Loops	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 3:

CLO 1:

Lab Goals/Objectives:

To understand the implementation of conditional iteration using while loops.

To apply while loops for handling dynamic user inputs, such as processing data for a specific number of students sequentially.

Equipment Required:

- Hardware: Laptop / PC
- Software: IDE (e.g., PyCharm / VS Code), Python/C Environment.

Theory:

While Loops: A while loop is a condition-controlled loop that repeatedly executes a block of code as long as a specified condition remains True. It is ideal when the exact number of iterations depends on runtime conditions or user input.

Loop Counters: To process a fixed number of entries (like a specific total number of students), a counter variable is initialized, checked in the loop condition, and incremented inside the loop body to avoid infinite execution.

Lab Work:

Task 1:

```
c = 1
while c > 0:
    name = input("Enter Student Name: ")
    roll_num = input("Enter Student Roll Number: ")

    a = int(input("Enter Obtained marks: "))
    b = (a * 100) / 300
    print(f"Percentage: {b:.2f}%")
    choice = input("Add another student? (y/n): ")
    if choice.lower() != "y":
        c = 0  # This breaks the loop
```

Result:

Conclusions:

This lab successfully demonstrated how while loops can be used to control program flow dynamically based on user-defined inputs. By automating a student grading system inside an iterative structure, a practical understanding of loop counters, condition testing, and repetitive data processing was achieved. All tasks executed correctly.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Manual Report	**Dated: 28/06/2026**
Semester: 1st	**Session: 2026-30**
Lab/Project/Assignment #: 4	**CLOs to be covered: CLO 2**
Lab Title: Nested Loops	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 4:

CLO 2:

Lab Goals/Objectives:

- To master the logic of nested loops (loop inside a loop) using variables like i and j to control rows and columns.
- To design and print complex geometric structures, including pyramids, inverse shapes, and diamond patterns, on the console screen.

Equipment Required:

- Hardware: Laptop / PC
- Software: IDE (e.g., PyCharm / VS Code), Python/C Environment.

Theory:

- Nested Loops: A nested loop consists of an outer loop and an inner loop. The outer loop (usually tracked by variable i) dictates the row number, while the inner loop (usually tracked by variable j) dictates what happens within that row, such as printing spaces or stars (*).
- Pattern Logic:
  Pyramids: Require the inner loop to calculate and print a decreasing number of leading spaces followed by an increasing, odd number of stars for each row.
  Diamonds: Created by combining two separate nested loop structures: an upright pyramid for the top half and an inverted pyramid for the bottom half. The loop boundaries change dynamically to decrease the width after reaching the center row.

Lab Work:

Task 1:

```
for i in range(1, 11):
    for j in range(1, 11):
        print(f"{i} * {j} = {i * j}")
    print()
#table (1s, 2s, 3s, etc.)
```

Result:

```
C:\Users\suvad\PycharmProjects\PythonProject11\.venv\Scripts\python.exe C:\Users\suvad\PycharmProjects
1 * 1 = 1
1 * 2 = 2
1 * 3 = 3
1 * 4 = 4
1 * 5 = 5
1 * 6 = 6
1 * 7 = 7
1 * 8 = 8
1 * 9 = 9
1 * 10 = 10
```

Task 2:

```
for i in range(1, 5):  # Outer loop controls the rows (1 to 4)
    for j in range(4 - i):
        print(" ", end="\t")
for k in range(2 * i - 1):
        print("*", end="\t")
    print()
```

Result:

```
C:\Users\suvad\PycharmProjects\PythonProject11\.venv\Scripts\python.exe C:\Users\suvad\PycharmProjects
        *
      *  *  *
    *  *  *  *  *
  *  *  *  *  *  *  *

Process finished with exit code 0
```

Task 3:

```
for i in range(1, 5):
    for j in range(1, 5):
```

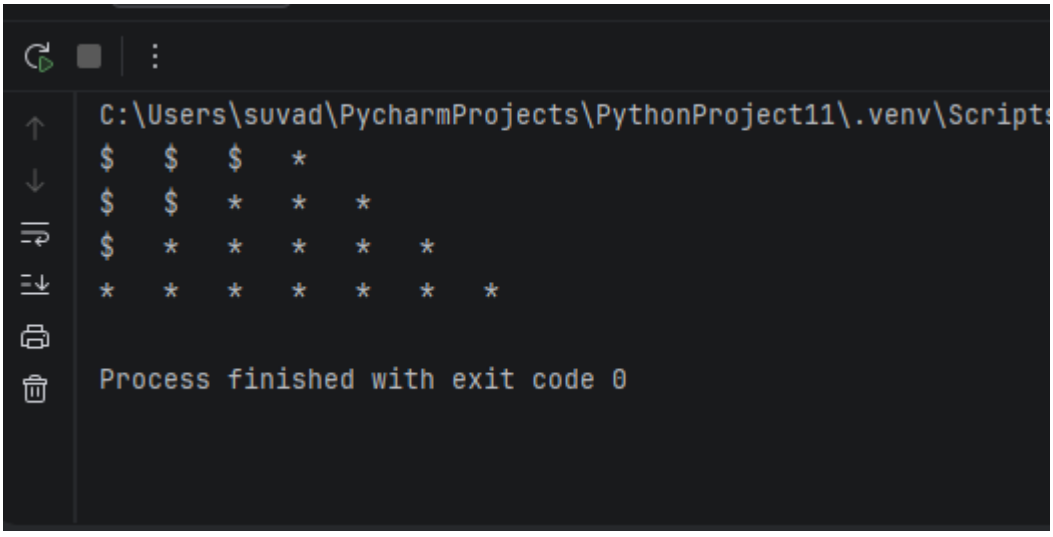
```
        print("*", end="\t")
    print()
```

Result:

Task 4:

```
for i in range(1, 5):
    for j in range(4 - i):
        print("$", end="\t")

    for k in range(2 * i - 1):
        print("*", end="\t")
    print()
```

Result:



Task 5:

```
for i in range(3, 0, -1):
    for j in range(4 - i):
        print(" ", end="\t")

    for k in range(2 * i - 1):
        print("*", end="\t")

    print()
```

Result:

Conclusions:

This lab successfully demonstrated the utility of for loops in handling repetitive tasks efficiently. By printing various star patterns, a solid understanding of nested loop logic, loop boundaries, and sequence iteration was achieved. All tasks executed with correct outputs.

University Of Engineering and Technology, Lahore **Computer Engineering Department**

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Manual Report	**Dated: 28/06/2026**
Semester: 1st	**Session: 2026-30**
Lab/Project/Assignment #: 5	**CLOs to be covered: CLO**
Lab Title: Functions	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**

(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 5:

CLO 2:

Lab Goals/Objectives:

To understand the concept of modular programming by breaking down code into reusable blocks called functions.

To master the implementation of function definitions, arguments/parameters passing, and return values.

Equipment Required:

- Hardware: Laptop / PC
- Software: IDE (e.g., PyCharm / VS Code), Python/C Environment.

Theory:

Functions: A function is a self-contained block of code designed to perform a specific task. It only runs when it is explicitly called. Functions help reduce code duplication and improve readability.
Parameters and Arguments: Information can be passed into functions as parameters, which act as variables inside the function definition.
Return Statements: A function can send data back to the main program using a return statement, allowing the result of a calculation to be stored or used elsewhere in the code.

Lab Work:

Task 1:

```
def hello():
    return
    print("Hello")


def info():
```

```
    return print(
        "Dawar and sohaib", "Roll Number:43,36", "Bank pin: 2218"
    )
```

Result:

Task 2:

```
def sum(a, b):
    return a + b


print(f"sum is:{sum(1, 2)}")
print(f"sum is:{sum(a=1, b=2)}")
```

Result:

Task 3:

```
print(len("Dawar"))

print(max(20, 20))

print(min(-20, 20))
```

```
print(sum([1, 6]))

print(type(1))
```

Result:

Task 4:

```
def person(name, roll):
    print(f"{name} How Are you! , roll number: {roll}")
    return name + " " + roll
person("Dawar", "50")
```

Result:

Conclusions:

This lab successfully demonstrated how to create and invoke user-defined functions to make code modular and efficient. By passing arguments and utilizing return values, a solid understanding of data scope and functional reusability was achieved. All code components executed correctly.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Manual Report	**Dated: 29/06/2026**
Semester: 1st	**Session: 2026-30**
Lab/Project/Assignment #: 6	**CLOs to be covered: CLO**
Lab Title: Local and Global Variables	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	**Marks**	**Level**	**Rubric**
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding

Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 6

CLO 2:

Lab Goals/Objectives:

To understand the concept of variable scope, specifically differentiating between local and global variables.

To learn how data accessibility and lifetime are managed across different parts of a program.

Equipment Required:

- Hardware: Laptop / PC
- Software: IDE (e.g., PyCharm / VS Code), Python/C Environment.

Theory:

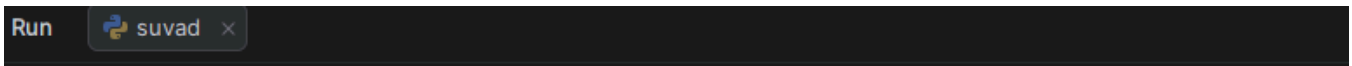
Scope: Scope determines the visibility and lifetime of a variable within a program's code.
Local Variables: Variables declared inside a function or a block are local. They are only accessible within that specific function and are destroyed once the function finishes executing.
Global Variables: Variables declared outside all functions (usually at the top of the script) are global. They can be accessed by any function throughout the entire program.
The global Keyword: If a function needs to modify a global variable (instead of just reading it), special keywords (like global in Python) must be used to grant write access.

Lab Work:

Task 1:

```
x = "25"
y = "Dawar"

print(x + y)


def welcome():
    x = 45
    print(x)
```

Result:



Task 2:

```
x = "25"
y = "Dawar"

def welcome():
    global x
    print(x)
    print(y)

welcome()
```

Result:

Task 3:

```
x = 25
def welcome():
    global x
    x = 45
    print(x)
welcome()
```

Result:

```
C:\Users\suvad\PycharmProjects\PythonProject11\.venv\Scripts\python.exe C:\Users\suvad
45

Process finished with exit code 0
```

Task 4:

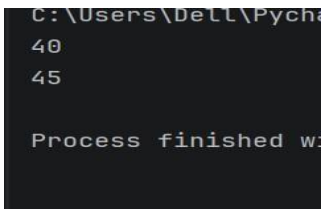
```
total_points = 45


def show_points():
    print(total_points)


show_points()
```

Result:

Task 5:

```
x = 45

def a():     x = 40
print(x)
print(globals()["x"])
```

Result:



Conclusions:

This lab successfully demonstrated the practical differences between local and global scopes. By managing variable accessibility across various functions, a solid understanding of memory lifetimes and data safety boundaries was achieved, ensuring proper control over variable isolation.

University Of Engineering and Technology, Lahore **Computer Engineering Department**

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Manual Report	**Dated: 28/06/2026**
Semester: 1st	**Session: 2026-30**
Lab/Project/Assignment #: 7	**CLOs to be covered: CLO 3**
Lab Title: List, Tuple	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 7

CLO 3:

Lab Goals/Objectives:

- To understand sequential data structures used to store collections of data in a single variable.
- To differentiate between mutable and immutable structures and learn when to apply lists versus tuples.

Equipment Required:

- Hardware: Laptop / PC
- Software: IDE (e.g., PyCharm / VS Code), Python/C Environment.

Theory:

- Lists: A list is an ordered collection of items that is mutable, meaning you can change, add, or remove elements after it has been created. It is defined using square brackets [].
- Tuples: A tuple is an ordered collection of items that is immutable, meaning its elements cannot be altered once assigned. It is defined using parentheses (). Tuples are faster than lists and protect data from accidental modification.
- Indexing: Both structures use zero-based indexing (e.g., data[0] accesses the first item), allowing efficient retrieval of stored elements.

Lab Work:

Task 1:

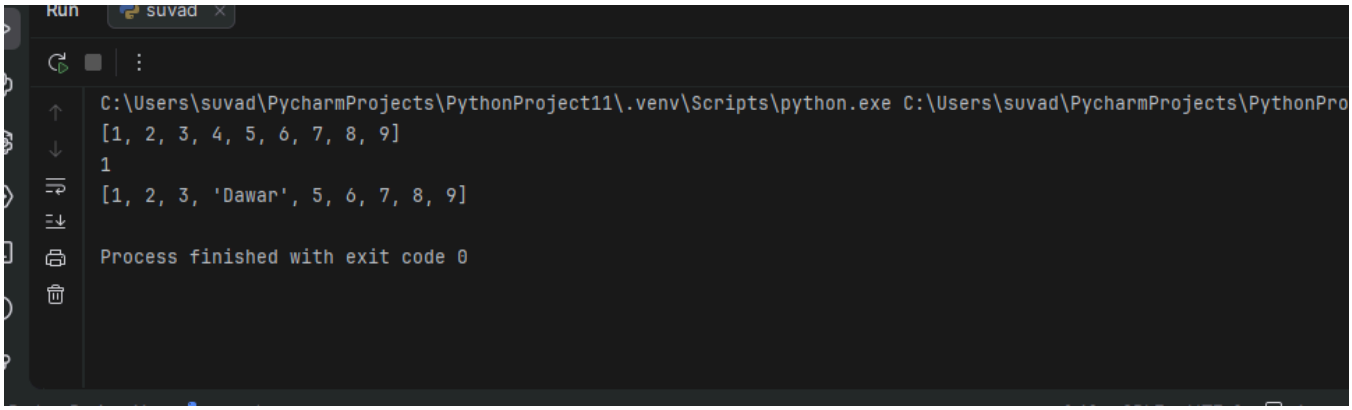
```
user_info = ["Dawar", "Imran", "CyberSecurity"]
print(len(user_info))
```

Result:

Task 2:

```
items = [1, 2, 3, 4, 5, 6, 7, 8, 9]
print(items)
print(items[0])

items[3] = "Dawar"
print(items)
```

Result:



Task 3:

```
data_list = [10, 20, "Dawar", "Zain", "Bilal"]
print(data_list)

# Adding an element to the end
data_list.append(50)
print(data_list)

# Removing a specific value
data_list.remove(10)
print(data_list)

# Removing an item by its index position
data_list.pop(2)
print(data_list)
```

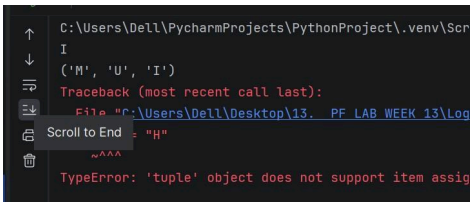
Result:

Task 4:(tuples indexing + checkinf if tupples are mutable or not)

```
initials = ("M", "D", "I")
print(initials[2])
```

```
print(initials)
temp_list = list(initials)
temp_list[1] = "S"
initials = tuple(temp_list)

print(initials)
```

Result:



Task 5:

```
t = ("M","U","I","H","M","M","U")
print(t.count("M"))
```

Result:

Conclusions:

This lab successfully demonstrated how to create, access, and manipulate data using lists and tuples. By comparing their behaviors, a clear understanding of mutability versus immutability was achieved, ensuring the correct choice of data structure depending on whether the collection requires dynamic updates or fixed data security.

University Of Engineering and Technology, Lahore **Computer Engineering Department**

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Manual Report	**Dated: 29/06/2026**
Semester: 1st	**Session: 2026-30**
Lab/Project/Assignment #: 8	**CLOs to be covered: CLO 3**
Lab Title: Sets, Dictionary	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 9

CLO 3:

Lab Goals/Objectives:

- To explore advanced built-in data structures for managing distinct collections and key-value pairs.
- To understand how to utilize sets for mathematical uniqueness and dictionaries for optimized data retrieval.

Equipment Required:

- Hardware: Laptop / PC
- Software: IDE (e.g., PyCharm / VS Code), Python/C Environment.

Theory:

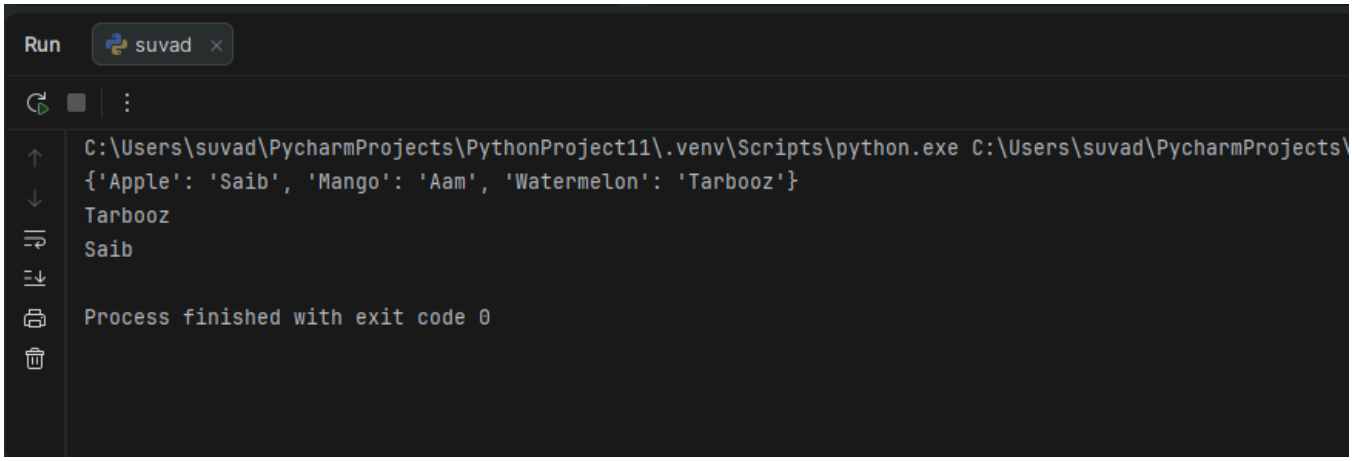
Sets: A set is an unordered collection of items where every element is unique (no duplicates allowed). Sets are mutable, but because they are unordered, they do not support indexing. They are highly efficient for operations like unions, intersections, and removing duplicate values from data.
Dictionaries: A dictionary is an ordered (as of recent language standards) collection of data stored as keyvalue pairs. Each key must be unique and acts as a custom index to access its associated value. Dictionaries are optimized for fast lookups, making it easy to retrieve data without searching through an entire collection.

Lab Work:

Task 1:

```
fruits_dict = {"Apple": "Saib", "Mango": "Aam",
"Watermelon": "Tarbooz"}
print(fruits_dict)

print(fruits_dict["Watermelon"])
print(fruits_dict["Apple"])
```

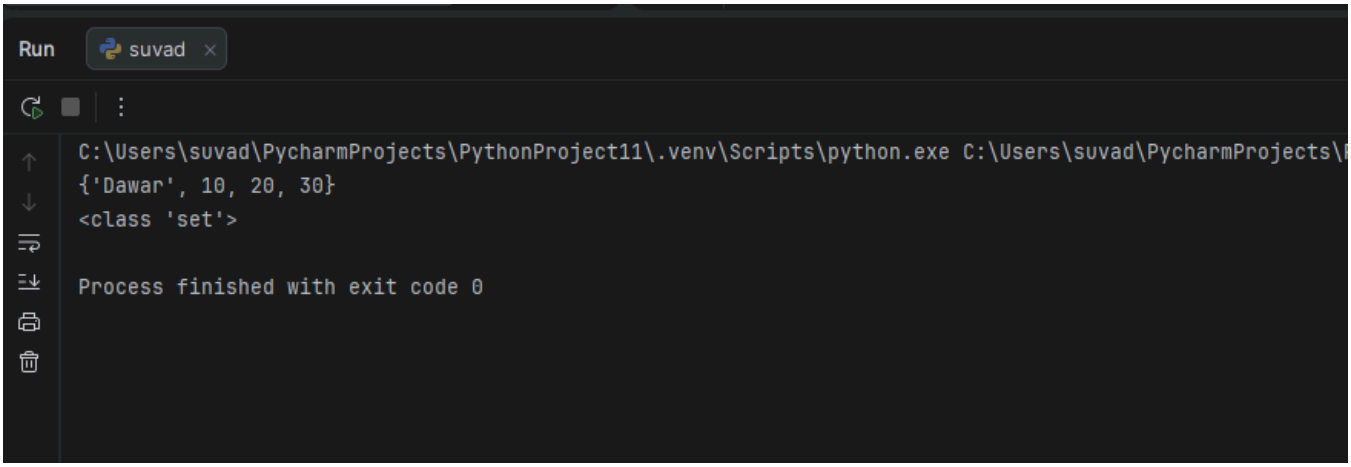
Result:



Task 2:

```
# Creating a set with unique values

unique_items = {10, 20, 30, "Dawar"}

print(unique_items)

empty_set = set()

print(type(empty_set))
```

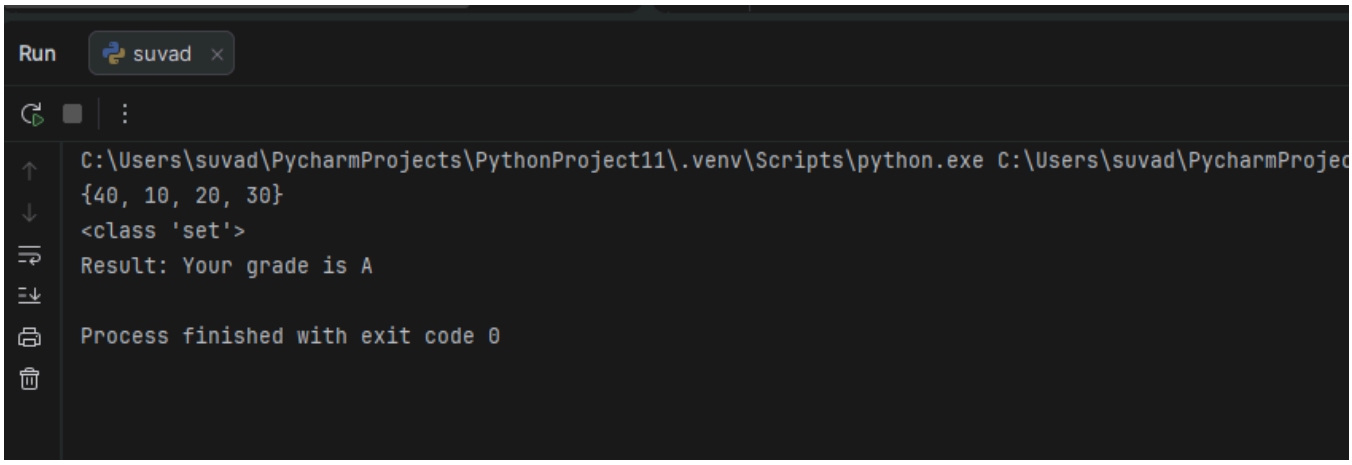
Result:



Task 3:

```
# Disguising the variable names and set content
numbers_set = set([10, 20, 30, 40])
print(numbers_set)
print(type(numbers_set))

print("Result: Your grade is A")
```

Result:



Task 4:

```
# Disguising the set names and
changing loop variables
course_codes = {"ce", "cs",
"cb", "ids"}
```

```
for course in course_codes:
    if "ce" == course:
        print("Computer
Engineering")
modified_courses = {"ce",
"cs", "cb", "ids"}
modified_courses.add("AE")
modified_courses.discard("cs")
modified_courses.discard("Me
")
```

Result:

Task 5:

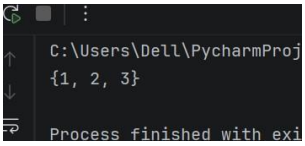
```
dept_codes = {"cs", "ce", "cb", "ids"}
dept_codes.remove("ce")
print(dept_codes)

dept_codes.pop()
print(dept_codes)

dept_codes.clear()
print(dept_codes)
```

Result:

Task 6:

```
a =set.union(s1,s2,s3)
 print(a)
```

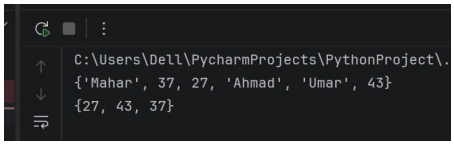
Result:



Task 7:

```
c = set.union(a,b)
 print(c)
```

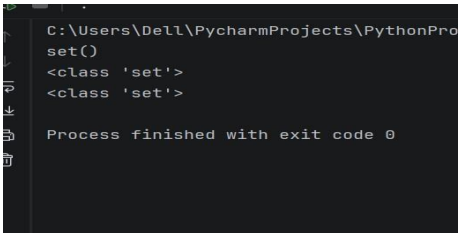
Result:



Task 8:

```
s1.update(s2)
print(s1)
 print(s2)
```

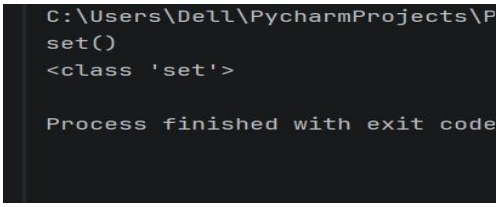
Result:



Task 9:

```
s = set.intersection(a,b)
a.intersection_update(b)
set.symmetric_difference(a,b)
```

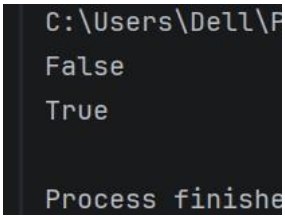
Result:



Task 10:

d = set.symmetric_difference(s1,s2)
print(d) s1 = {1,2,3} s2 ={4,5,3} a
=s1.difference(s2)
print(s1.isdisjoint(s2))

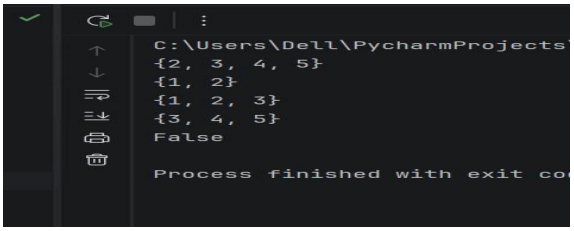
Result:



Task 11:

s1 = {1,2,3,4} s2
= {1,2,3}
print(set.issubset(s1,s2))
print(set.issubset(s2,s1))

Result:



Conclusions:

This lab successfully demonstrated the practical applications of sets and dictionaries. By implementing sets, a clear understanding of handling unique data without duplicates was achieved. Furthermore, working with dictionaries provided hands-on experience in structuring data logically using keys, enabling highly efficient and readable code configurations.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Manual Report	**Dated: 28/06/2026**
Semester: 1st	**Session: 2026-30**
Lab/Project/Assignment #: 10	**CLOs to be covered: CLO 2**
Lab Title: Recursion	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 10

CLO 2:

Lab Goals/Objectives:

- To understand the concept of self-referential programming logic through recursive functions.
- To master the design of base cases and recursive steps to solve complex algorithmic problems efficiently.

Equipment Required:

- Hardware: Laptop / PC
- Software: IDE (e.g., PyCharm / VS Code), Python/C Environment.

Theory:

- Recursion: Recursion is a programming technique where a function calls itself directly or indirectly to solve a smaller instance of the same problem.
- Base Case: The crucial condition that stops the recursion from executing infinitely. When the base case is met, the function starts returning values back up the call stack.
- Recursive Step: The part of the function where the core logic is applied, and the function calls itself with modified parameters, gradually moving closer to the base case.

Lab Work:

Task 1:

```
def
display_val(n
um):
   if num ==
0:
      return
   print(num)

display_val(n
um - 1)

display_val(5
)
```

Result:

Task 2:

```
def factorial(n): if n==0 or
n==1: return 1 else:
return n*factorial(n-1)
```

Result:

Task 3:

```
def get_fib(num):
    if num <= 0:
        return 0
    if num == 1:
        return 1
    return get_fib(num - 1) + get_fib(num - 2)


print(get_fib(6))
```

Result:

Conclusions:

This lab successfully demonstrated how recursion breaks down complex, repetitive problems into simpler sub-problems. By implementing recursive functions, a solid understanding of the system call stack, memory management, and the absolute necessity of a well-defined base case was achieved.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Manual Report	**Dated: 28/06/2026**
Semester: 1st	**Session: 2026-30**
Lab/Project/Assignment #: 11	**CLOs to be covered: CLO 4**
Lab Title: Exception Handling	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 11

CLO 4:

Lab Goals/Objectives:

- To understand how to handle runtime errors gracefully without causing the program to crash.
- To master the implementation of error-trapping blocks (try-except) to build robust and faulttolerant applications.

Equipment Required:

- Hardware: Laptop / PC
- Software: IDE (e.g., PyCharm / VS Code), Python/C Environment.

Theory:

- Exceptions: Exceptions are unexpected errors that occur during program execution (e.g., ZeroDivisionError, ValueError, or FileNotFoundError). If unhandled, they terminate the program immediately.
- The Try-Except Block: * try: Encloses the code that might potentially throw an error.
- except: Catches and handles specific errors if they occur inside the try block, allowing the program to continue running safely.
- Additional Clauses: * else: Executes only if no exceptions were raised in the try block.
- finally: Executes unconditionally at the end, regardless of whether an error occurred or not (typically used for cleanup tasks like closing files).

Lab Work:

Task 1:

```
try:
    user_num = int(input("Enter any Number: "))
    print(user_num)
except ValueError:
    print("InValid")
print("Hello World")
```

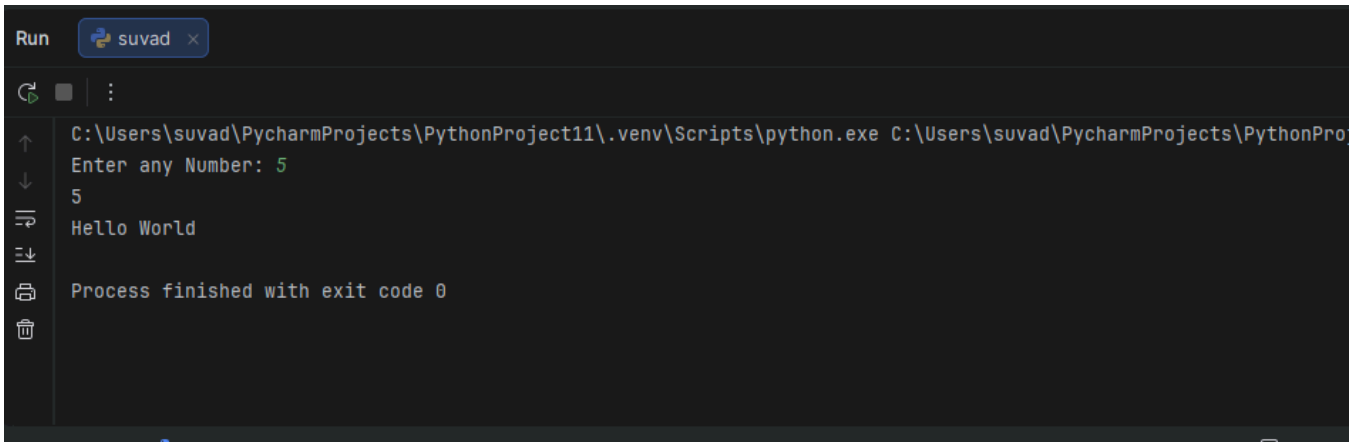
Result:

Task 2:

```
try:
    user_val = int(input("Enter any Number: "))
    print(user_val)
```

```
except Exception as error_msg:
    print(error_msg)
print("Hello World")
```

Result:



Conclusions:

This lab successfully demonstrated the critical role of exception handling in preventing runtime failures. By intercepting potential faults dynamically, a practical understanding of program stability was achieved, ensuring that user errors do not disrupt the continuous flow of execution.

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Manual Report	**Dated: 28/06/2026**
Semester: 1st	**Session: 2026-30**
Lab/Project/Assignment #: 12	**CLOs to be covered: CLO 2**
Lab Title: Lambda Functions	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 12

CLO 2:

Lab Goals/Objectives:

- To understand the creation and application of anonymous, single-line functions.
- To learn how to use lambda functions alongside higher-order functions like map(), filter(), and reduce() for cleaner code.

Equipment Required:

- Hardware: Laptop / PC
- Software: IDE (e.g., PyCharm / VS Code), Python/C Environment.

Theory:

- Lambda Functions: A lambda function is a small, anonymous function that is defined without a name using the lambda keyword. Unlike standard functions defined with def, lambda functions can only contain a single expression and automatically return its result.
- Syntax: The basic structure is lambda arguments: expression. They can take any number of arguments but can only have one execution step.
- Inline Utility: They are primarily used as short, temporary throwaway functions where creating a full named function would be unnecessary overhead, making the codebase highly concise.

Lab Work:

Task 1:

```
square_num = lambda val: val * val
print(square_num(5))
```

Result:

Task 2:

```
calc_sum = lambda first, second: first + second

print(calc_sum(2, 3))
```

Result:

Task 3:

```
calc_total = lambda first, second, third: first + second
+ third
print(calc_total(2, 3, 2))
```

Result:

Conclusions:

This lab successfully demonstrated the utility of lambda functions in writing compact and readable code. By implementing these anonymous expressions, a solid understanding of functional programming concepts was achieved, showing how inline operations can optimize short computational workflows.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Manual Report	**Dated: 28/06/2026**
Semester: 1st	**Session: 2026-30**
Lab/Project/Assignment #: 13	**CLOs to be covered: CLO 2**
Lab Title: MAP, Filter and Reduce	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**

(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 13

CLO 1:

Lab Goals/Objectives:

- To understand the paradigm of functional programming through higher-order utilities.
- To master processing data collections efficiently without using explicit for-loops by using map(), filter(), and reduce().

Equipment Required:

- Hardware: Laptop / PC
- Software: IDE (e.g., PyCharm / VS Code), Python/C Environment.

Theory:

- Higher-Order Functions: These are functions that take another function as an argument or return a function as a result.
- map(function, iterable): Applies a specified function to every item in an iterable (like a list) and returns a new iterator containing the results.
- filter(function, iterable): Tests each element in an iterable against a function that returns a boolean. It constructs an iterator from those elements that evaluate to True.
- reduce(function, iterable): (Imported from the functools module) Continually applies a function to the sequence pairs elements from left to right, reducing the entire collection down to a single cumulative value.

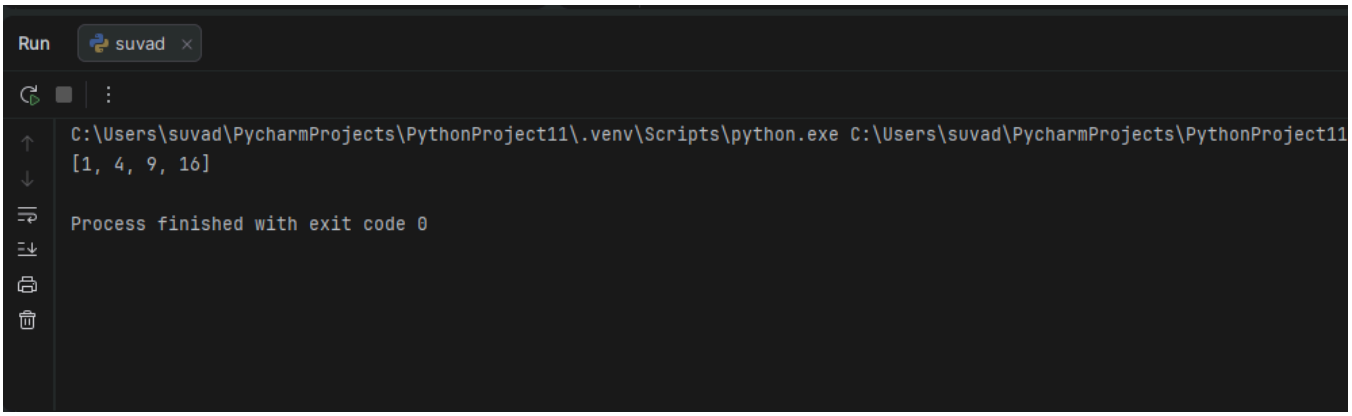
Lab Work:

Task 1:

```
numbers_list = [1, 2, 3, 4]
```

```
squared_list = list(map(lambda num: num * num,
numbers_list))
print(squared_list)
```

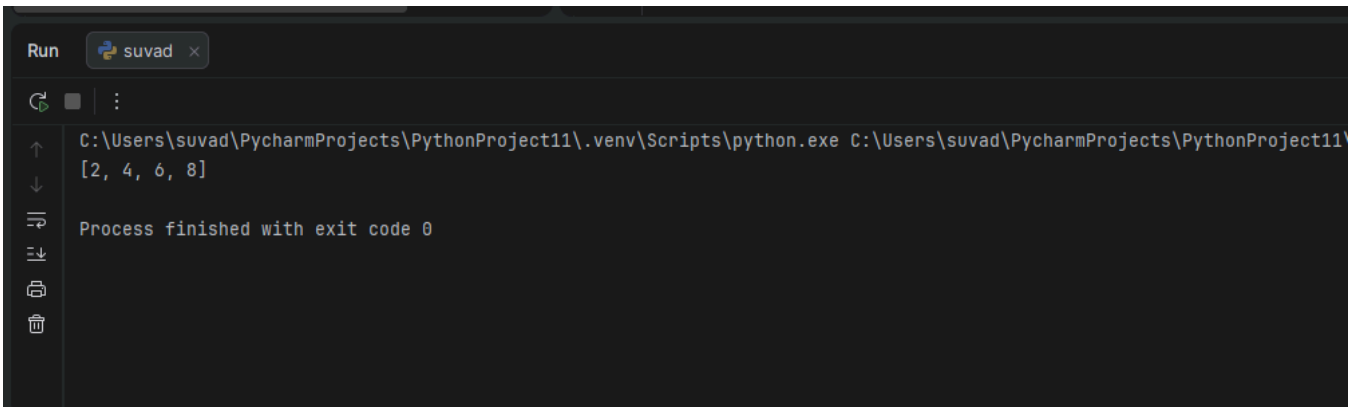
Result:



Task 2:

```
data_list = [1, 2, 3, 4]
doubled_data = list(map(lambda val: val * 2, data_list))
print(doubled_data)
```

Result:



Conclusions:

This lab successfully demonstrated the power of functional programming arrays in streamlining data processing workflows. By replacing manual iteration loops with map, filter, and reduce, a solid understanding of memory efficiency, functional transformations, and cleaner code structures was achieved.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Manual Report	**Dated: 28/06/2026**
Semester: 1st	**Session: 2026-30**
Lab/Project/Assignment #: 14	**CLOs to be covered: CLO 2**
Lab Title: OS Library, if __name__ == "__main__", is vs ==	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 14

CLO 2:

Lab Goals/Objectives:

- To explore system-level operations using the os library for directory and file management.
- To understand program entry points using if __name__ == '__main__' and differentiate between identity and value comparison operators (is vs ==).

Equipment Required:

- Hardware: Laptop / PC
- Software: IDE (e.g., PyCharm / VS Code), Python/C Environment.

Theory:

- The os Library: A built-in Python module that provides functions to interact with the operating system. It allows programs to perform tasks like creating, renaming, or deleting folders, fetching environment variables, and navigating file paths.
- if __name__ == '__main__': This construct acts as the script's entry point. The special variable __name__ gets set to "__main__" only when the script is run directly. If the script is imported as a module into another file, the code inside this block will not execute, preventing accidental execution.
- is vs == Operators: * == (Equality): Compares the values of two objects to see if they are equal.

is (Identity): Compares the memory addresses of two objects to see if they point to the exact same place in memory.

Lab Work:

Task 1:

```
def greet_first():
   print("Welcome to class!!")


def greet_second():
   print("Welcome to class2!!")
```

```
if __name__ == "__main__":
    greet_first()
    greet_second()
```

Result:

Task 2:

```
def greet_user():
    print("Welcome to class!!")


def greet_user_two():
    print("Welcome to class2!!")


if __name__ == "__main__":
    greet_user()
```

Result:

Conclusions:

This lab successfully demonstrated system interactions and advanced conditional structures in Python. By utilizing the os module, direct control over file environments was achieved. Furthermore, mastering the execution entry point and the structural difference between object value (==) and memory allocation (is) provided essential insights into Python's internal execution logic.

University Of Engineering and Technology, Lahore **Computer Engineering Department**

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Manual Report	**Dated: 29/06/2026**
Semester: 1st	**Session: 2026-30**
Lab/Project/Assignment #: 156	**CLOs to be covered: CLO 3**
Lab Title: GUI using Qt Designer	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	**Marks**	**Level**	**Rubric**
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.

Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 15

CLO 3:

Lab Goals/Objectives:

- To understand the principles of Graphical User Interface (GUI) design and event-driven programming.
- To master the layout creation using Qt Designer and integrate the resulting interface with a Python backend script.

.

Equipment Required:

- Hardware: Laptop / PC
- Software: IDE (e.g., PyCharm / VS Code), Python/C Environment.

Theory:

- GUI vs. CLI: A Graphical User Interface (GUI) provides visual components like buttons, text fields, and menus, allowing users to interact with software intuitively, unlike command-line interfaces.
- Qt Designer: A visual layout tool provided by the Qt framework. It allows developers to drag and drop widgets (such as QPushButton, QLabel, and QLineEdit) to build layouts visually, which saves the interface configuration as a .ui file (XML format).
- Signals and Slots: This is Qt's mechanism for event handling. When a user interacts with a widget (e.g., clicking a button), a Signal is emitted. This signal is connected to a Slot (a backend Python function) that executes the corresponding logic.
- UI Conversion: The visual .ui file is integrated into Python either by converting it into a .py file using the pyuic5 tool or by loading it dynamically at runtime using the uic.loadUi() method.

Lab Work:

Task 1:

```
import sys, os, json, calendar

from datetime import datetime

from PySide6.QtWidgets import (QApplication,
QWidget, QLabel, QGridLayout, QPushButton,

    QHBoxLayout, QVBoxLayout, QFrame,
QMessageBox, QMenu, QDialog, QLineEdit,
QTextEdit,
```

```
    QComboBox, QDialogButtonBox,
QFormLayout, QTimeEdit, QScrollArea,
QToolButton,

    QGraphicsOpacityEffect, QSizePolicy)

from PySide6.QtGui import (QPixmap, QColor,
QCursor, QPainter, QLinearGradient, QIcon,
QAction,

    QDesktopServices, QFont, QRadialGradient,
QConicalGradient, QPainterPath, QPen, QBrush,

    QFontMetrics)

from PySide6.QtCore import (Qt, QTimer,
QTime, QUrl, QPropertyAnimation,
QEasingCurve,

    QParallelAnimationGroup,
QSequentialAnimationGroup, QRect, QRectF,
QPoint, QPointF,

    Property, QObject)



DIR =
os.path.dirname(os.path.abspath(__file__))

IMG_DIR = os.path.join(DIR, "images")

DATA_FILE = os.path.join(DIR,
"calendar_events.json")

IMG_NAMES =
["1january.jpg","2february.jpg","3march.jpg","4a
pril.jpg","5may.jpg","6june.jpg",


"7july.jpg","8august.jpg","9september.jpg","10oc
tober.jpg","11november.jpg","12december.jpg"]

GRADIENTS =
[("#3a6186","#89253e"),("#1f4037","#99f2c8"),(
"#355c7d","#6c5b7b"),("#56ab2f","#a8e063"),


("#f7971e","#ffd200"),("#11998e","#38ef7d"),("#
fc4a1a","#f7b733"),("#1488cc","#2b32b2"),


("#ff5f6d","#ffc371"),("#4b6cb7","#182848"),("#
834d9b","#d04ed6"),("#0f2027","#2c5364")]
```

```
COLORS =
{"Tomato":"#d50000","Flamingo":"#e67c73","T
angerine":"#f4511e","Banana":"#f6bf26",


"Sage":"#33b679","Basil":"#0b8043","Peacock":
"#039be5","Blueberry":"#3f51b5",


"Lavender":"#7986cb","Grape":"#8e24aa","Grap
hite":"#616161"}

WD =
["Mon","Tue","Wed","Thu","Fri","Sat","Sun"]

WD_MINI = ["M","T","W","T","F","S","S"]



PALETTES = {

   "light": dict(bg="#ffffff", bg_alt="#f6f8fc",
border="#dadce0", text="#202124",

                text_dim="#5f6368",
accent="#1a73e8", hover="#f1f3f4",
weekend="#d93025"),

   "dark": dict(bg="#202124",
bg_alt="#26272b", border="#3c4043",
text="#e8eaed",

               text_dim="#9aa0a6",
accent="#8ab4f8", hover="#33343a",
weekend="#f28b82"),

}



def system_theme():

   try:

      c = QApplication.palette().window().color()

      return "dark" if
(0.299*c.red()+0.587*c.green()+0.114*c.blue())
< 128 else "light"

   except Exception:

      return "light"


def holidays(y):
```

```
    return {f"{y}-01-01":"New Year's Day",
f"{y}-03-08":"Women's Day",
f"{y}-03-23":"Pakistan Day",

            f"{y}-04-22":"Earth Day",
f"{y}-05-01":"Labour Day", f"{y}-05-08":"Red
Cross Day",

            f"{y}-06-05":"Environment Day",
f"{y}-07-01":"Doctor's Day",
f"{y}-08-14":"Independence Day",

            f"{y}-09-06":"Defence Day",
f"{y}-10-02":"Gandhi's Birthday",
f"{y}-11-09":"Iqbal Day",

            f"{y}-12-10":"Human Rights Day"}


def fmt_time(t):

    try: return datetime.strptime(t,
"%H:%M").strftime("%I:%M %p").lstrip("0")

    except Exception: return t



#
═══════════════════════════════════════════
═══════════════════════════════════════════
═══════════════════════════════

#  SPLASH SCREEN

#
═══════════════════════════════════════════
═══════════════════════════════════════════
═══════════════════════════════



class AnimatedOrb(QWidget):

    """Floating translucent orb for the splash
background."""

    def __init__(self, parent, x, y, size, color1,
color2, speed=3000):

        super().__init__(parent)

        self.setGeometry(x, y, size, size)
```

```
self.setAttribute(Qt.WA_TransparentForMouseE
vents)


self.setAttribute(Qt.WA_TranslucentBackground
)
        self._color1 = QColor(color1)
        self._color2 = QColor(color2)
        self._offset = 0.0
        self._size = size
        self._timer = QTimer(self)
        self._timer.timeout.connect(self._animate)
        self._timer.start(16)
        self._t = 0
        self._speed = speed
        self._ox, self._oy = x, y
        self._amp = 18


    def _animate(self):
        import math
        self._t += 16
        dx = math.sin(self._t / self._speed * 2 *
3.14159) * self._amp
        dy = math.cos(self._t / self._speed * 2 *
3.14159) * self._amp * 0.6
        self.move(int(self._ox + dx), int(self._oy +
dy))
        self.update()


    def paintEvent(self, e):
        p = QPainter(self)
        p.setRenderHint(QPainter.Antialiasing)
        grad = QRadialGradient(self._size/2,
self._size/2, self._size/2)
        c1 = QColor(self._color1); c1.setAlpha(90)
```

```
        c2 = QColor(self._color2); c2.setAlpha(0)
        grad.setColorAt(0, c1)
        grad.setColorAt(1, c2)
        p.setBrush(QBrush(grad))
        p.setPen(Qt.NoPen)
        p.drawEllipse(0, 0, self._size, self._size)


class CalendarIconWidget(QWidget):
    """Animated mini calendar icon with a
glowing ring."""
    def __init__(self, parent=None):
        super().__init__(parent)
        self.setFixedSize(110, 110)
        self._angle = 0
        self._t = QTimer(self)
        self._t.timeout.connect(self._spin)
        self._t.start(16)
        self._rot = 0

    def _spin(self):
        self._rot = (self._rot + 0.5) % 360
        self.update()

    def paintEvent(self, e):
        p = QPainter(self)
        p.setRenderHint(QPainter.Antialiasing)
        cx, cy, r = 55, 55, 46

        # Spinning gradient ring
        grad = QConicalGradient(cx, cy, self._rot)
        grad.setColorAt(0.0, QColor("#1a73e8"))
```

```
        grad.setColorAt(0.3, QColor("#4285f4"))
        grad.setColorAt(0.6, QColor("#8ab4f8"))
        grad.setColorAt(1.0, QColor("#1a73e8"))
        pen = QPen(QBrush(grad), 4)
        p.setPen(pen)
        p.setBrush(Qt.NoBrush)
        p.drawEllipse(cx - r, cy - r, r * 2, r * 2)

        # Soft halo behind the card
        p.setPen(Qt.NoPen)
        p.setBrush(QColor(26, 115, 232, 18))
        p.drawEllipse(cx - r + 6, cy - r + 6, (r - 6) *
2, (r - 6) * 2)

        # Calendar emoji-style icon
        inner = r - 10
        rect = QRectF(cx - inner, cy - inner, inner *
2, inner * 2)
        p.setBrush(QColor(255, 255, 255))
        p.setPen(QPen(QColor(218, 220, 224), 1.5))
        p.drawRoundedRect(rect, 6, 6)

        # Calendar header bar
        header = QRectF(cx - inner, cy - inner, inner
* 2, inner * 0.55)
        p.setBrush(QColor(26, 115, 232, 230))
        p.setPen(Qt.NoPen)
        p.drawRoundedRect(header, 6, 6)
        p.drawRect(QRectF(cx - inner, cy - inner +
header.height() - 6, inner * 2, 6))

        # Day number in center
        now = datetime.now()
```

```
        p.setPen(QColor(32, 33, 36))
        f = QFont("Segoe UI", 14, QFont.Bold)
        p.setFont(f)
        p.drawText(QRectF(cx - inner, cy - inner +
inner * 0.6, inner * 2, inner * 1.2),
                   Qt.AlignHCenter | Qt.AlignTop,
str(now.day))



class SplashScreen(QWidget):
    """Full-screen animated splash / launch
screen."""

    def __init__(self, on_enter):
        super().__init__()
        self._on_enter = on_enter

self.setWindowFlags(Qt.FramelessWindowHint |
Qt.Window)
        self.setFixedSize(1000, 640)

        # Background
        self.setAttribute(Qt.WA_StyledBackground,
True)
        self.setStyleSheet("background: #ffffff;")

        # Decorative orbs
        AnimatedOrb(self, -60, -60, 340, "#1a73e8",
"#8ab4f8", 5000)
        AnimatedOrb(self, 700, 400, 280,
"#4285f4", "#aecbfa", 7000)
        AnimatedOrb(self, 300, 500, 200,
"#669df6", "#c2e7ff", 4000)
        AnimatedOrb(self, 820, -40, 220,
"#8ab4f8", "#d2e3fc", 6000)
```

```
        # Main layout
        lay = QVBoxLayout(self)
        lay.setContentsMargins(0, 0, 0, 0)
        lay.setSpacing(0)

        # Top spacer
        lay.addStretch(2)

        # Icon + wordmark row
        center = QHBoxLayout()
        center.addStretch()

        v_center = QVBoxLayout()
        v_center.setSpacing(0)
        v_center.setAlignment(Qt.AlignHCenter)

        # Calendar icon
        icon_row = QHBoxLayout()
        icon_row.addStretch()
        self._icon = CalendarIconWidget(self)
        icon_row.addWidget(self._icon)
        icon_row.addStretch()
        v_center.addLayout(icon_row)

        v_center.addSpacing(22)

        # App name
        self._name_label = QLabel("Pro Calendar")

self._name_label.setAlignment(Qt.AlignCenter)
        font = QFont("Segoe UI", 48, QFont.Light)
```

```
font.setLetterSpacing(QFont.AbsoluteSpacing,
-1.5)
        self._name_label.setFont(font)
        self._name_label.setStyleSheet("color:
#202124;")
        v_center.addWidget(self._name_label)


        v_center.addSpacing(8)


        # Tagline
        self._tag_label = QLabel("Your year,
beautifully organized.")

self._tag_label.setAlignment(Qt.AlignCenter)
        tag_font = QFont("Segoe UI", 13)

tag_font.setLetterSpacing(QFont.AbsoluteSpacin
g, 2.5)
        self._tag_label.setFont(tag_font)
        self._tag_label.setStyleSheet("color:
rgba(26,115,232,0.85);")
        v_center.addWidget(self._tag_label)


        v_center.addSpacing(48)


        # Enter button
        btn_row = QHBoxLayout()
        btn_row.addStretch()
        self._enter_btn = QPushButton("  Open
Calendar  →")
        self._enter_btn.setFixedSize(220, 52)

self._enter_btn.setCursor(QCursor(Qt.PointingH
andCursor))
```

```
        self._enter_btn.setFont(QFont("Segoe UI",
12, QFont.Medium))
        self._enter_btn.setStyleSheet("""
            QPushButton {
                background:
qlineargradient(x1:0,y1:0,x2:1,y2:0,
                    stop:0 #1a73e8, stop:1 #4285f4);
                color: white;
                border: none;
                border-radius: 26px;
                letter-spacing: 0.5px;
            }
            QPushButton:hover {
                background:
qlineargradient(x1:0,y1:0,x2:1,y2:0,
                    stop:0 #1765cc, stop:1 #3367d6);
            }
            QPushButton:pressed {
                background:
qlineargradient(x1:0,y1:0,x2:1,y2:0,
                    stop:0 #135cb0, stop:1 #2a56b8);
            }
        """)

self._enter_btn.clicked.connect(self._launch)
        btn_row.addWidget(self._enter_btn)
        btn_row.addStretch()
        v_center.addLayout(btn_row)


        center.addLayout(v_center)
        center.addStretch()
        lay.addLayout(center)


        lay.addStretch(2)
```

```
        # Bottom strip
        bottom = QHBoxLayout()
        bottom.setContentsMargins(40, 0, 40, 28)
        now = datetime.now()
        month_names =
["January","February","March","April","May","J
une",

"July","August","September","October","Novem
ber","December"]
        self._date_label = QLabel(
            f"{month_names[now.month-1].upper()}
·  {now.year}"
        )
        self._date_label.setFont(QFont("Segoe UI",
9))
        self._date_label.setStyleSheet("color:
rgba(32,33,36,0.35); letter-spacing: 3px;")

        self._ver_label = QLabel("v 2026")
        self._ver_label.setFont(QFont("Segoe UI",
9))
        self._ver_label.setStyleSheet("color:
rgba(32,33,36,0.35); letter-spacing: 2px;")
        bottom.addWidget(self._date_label)
        bottom.addStretch()
        bottom.addWidget(self._ver_label)
        lay.addLayout(bottom)

        # Fade-in effect
        self._opacity_effect =
QGraphicsOpacityEffect(self)
        self.setGraphicsEffect(self._opacity_effect)
        self._opacity_effect.setOpacity(0)
```

```
        self._fade_in =
QPropertyAnimation(self._opacity_effect,
b"opacity")
        self._fade_in.setStartValue(0.0)
        self._fade_in.setEndValue(1.0)
        self._fade_in.setDuration(800)

self._fade_in.setEasingCurve(QEasingCurve.Out
Cubic)
        QTimer.singleShot(80, self._fade_in.start)


    def _launch(self):
        self._fade_out =
QPropertyAnimation(self._opacity_effect,
b"opacity")
        self._fade_out.setStartValue(1.0)
        self._fade_out.setEndValue(0.0)
        self._fade_out.setDuration(400)

self._fade_out.setEasingCurve(QEasingCurve.In
Cubic)
        self._fade_out.finished.connect(self._finish)
        self._fade_out.start()


    def _finish(self):
        self.hide()
        self._on_enter()
        self.deleteLater()


    def keyPressEvent(self, e):
        if e.key() in (Qt.Key_Return, Qt.Key_Enter,
Qt.Key_Space):
            self._launch()
        super().keyPressEvent(e)
```

```
# ═══════════════════════════════════════════════════════════════════════
# ORIGINAL CALENDAR CODE (unchanged logic)
# ═══════════════════════════════════════════════════════════════════════

class Event:
    def __init__(self, title, time="", color="Peacock", notes="", link=""):
        self.title, self.time, self.color, self.notes, self.link = title, time, color, notes, link
    def to_dict(self): return self.__dict__
    @staticmethod
    def from_dict(d): return Event(d.get("title",""), d.get("time",""), d.get("color","Peacock"), d.get("notes",""), d.get("link",""))

class EventDialog(QDialog):
    def __init__(self, parent, date_str, existing=None):
        super().__init__(parent)
        self.setWindowTitle("Event"); self.setMinimumWidth(380)
        self.result_event, self.delete_requested = None, False
        lay = QVBoxLayout(self)
        lay.addWidget(QLabel(datetime.strptime(date_str, "%Y-%m-%d").strftime("%A, %B %d, %Y")))
        form = QFormLayout()
```

```
        self.title_edit = QLineEdit(existing.title if
existing else "")

        self.title_edit.setPlaceholderText("Add title
(e.g. Birthday: Sara)")

        form.addRow("Title", self.title_edit)



        row = QHBoxLayout()

        self.all_day = QComboBox();
self.all_day.addItems(["All day", "At a time"])

        self.time_edit = QTimeEdit();
self.time_edit.setDisplayFormat("hh:mm AP")

        if existing and existing.time:

            self.all_day.setCurrentIndex(1)

            h, m = map(int, existing.time.split(":"));
self.time_edit.setTime(QTime(h, m))

        else:

            self.time_edit.setTime(QTime(9, 0));
self.time_edit.setEnabled(False)


self.all_day.currentIndexChanged.connect(lambd
a i: self.time_edit.setEnabled(i == 1))

        row.addWidget(self.all_day);
row.addWidget(self.time_edit)

        form.addRow("When", row)



        self.color_combo = QComboBox()

        for name, hexcode in COLORS.items():

            self.color_combo.addItem(name)

            pix = QPixmap(14, 14);
pix.fill(QColor(hexcode))


self.color_combo.setItemIcon(self.color_combo.c
ount()-1, QIcon(pix))

        if existing:

            i =
self.color_combo.findText(existing.color)
```

```
            if i >= 0:
self.color_combo.setCurrentIndex(i)

        form.addRow("Color", self.color_combo)


        self.notes_edit = QTextEdit(existing.notes if
existing else "")

        self.notes_edit.setFixedHeight(70)

        form.addRow("Notes", self.notes_edit)


        self.link_edit = QLineEdit(existing.link if
existing else "")

        self.link_edit.setPlaceholderText("https://...
(Zoom, registration page, etc.)")

        form.addRow("Link", self.link_edit)

        lay.addLayout(form)


        btns = QHBoxLayout()

        if existing:

            del_btn = QPushButton("Delete event");
del_btn.clicked.connect(self._delete)

            btns.addWidget(del_btn)

        btns.addStretch()

        box =
QDialogButtonBox(QDialogButtonBox.Cancel |
QDialogButtonBox.Save)

        box.accepted.connect(self._save);
box.rejected.connect(self.reject)

        btns.addWidget(box); lay.addLayout(btns)


    def _save(self):

        title = self.title_edit.text().strip()

        if not title:

            QMessageBox.warning(self, "Missing
title", "Please give the event a title.")

            return
```

```
        t = self.time_edit.time().toString("HH:mm")
if self.all_day.currentIndex() == 1 else ""
        link = self.link_edit.text().strip()
        if link and not
link.lower().startswith(("http://", "https://")):
            link = "https://" + link
        self.result_event = Event(title, t,
self.color_combo.currentText(),
self.notes_edit.toPlainText().strip(), link)
        self.accept()

    def _delete(self):
        self.delete_requested = True
        self.accept()



class DayCell(QFrame):
    MAX_CHIPS = 3
    def __init__(self, owner):
        super().__init__()
        self.owner, self.date_key = owner, None
        self.setMinimumSize(90, 78);
self.setCursor(QCursor(Qt.PointingHandCursor))
        v = QVBoxLayout(self);
v.setContentsMargins(6, 4, 6, 4); v.setSpacing(2)
        top = QHBoxLayout()
        self.num = QLabel("");
self.num.setFixedSize(26, 26);
self.num.setAlignment(Qt.AlignCenter)
        top.addWidget(self.num); top.addStretch()
        v.addLayout(top)
        self.chips = QVBoxLayout();
self.chips.setSpacing(2)
        v.addLayout(self.chips); v.addStretch()
        self.p = PALETTES["light"]
```

```
    def _clear_chips(self):
        while self.chips.count():
            w = self.chips.takeAt(0).widget()
            if w: w.deleteLater()

    def _chip(self, text, color):
        l = QLabel(text)
        l.setStyleSheet(f"background:{color};
color:white; border-radius:4px; font-size:10px;
padding:2px 6px;")
        l.setMaximumWidth(160)
        return l

    def set_empty(self, p):
        self.p = p; self.date_key = None;
self.num.setText(""); self.num.setStyleSheet("")
        self._clear_chips(); self.setEnabled(False)

self.setStyleSheet(f"background:{p['bg_alt']};
border:1px solid {p['border']};")

    def set_day(self, day, evs, holiday, is_today,
is_weekend, date_key, p):
        self.p, self.date_key = p, date_key
        self.setEnabled(True);
self.num.setText(str(day))
        if is_today:

self.num.setStyleSheet(f"background:{p['accent']
}; color:white; border-radius:13px;
font-weight:700;")
        else:

self.num.setStyleSheet(f"color:{p['weekend'] if
is_weekend else p['text']}; font-weight:600;")
```

```
        self.setStyleSheet(f"background:{p['bg']};
border:1px solid {p['border']};")

        self._clear_chips()

        if holiday and not evs:

            self.chips.addWidget(self._chip(holiday,
"#0b8043"))

        else:

            shown = evs[:self.MAX_CHIPS]

            for ev in shown:

                label = ev.title if not ev.time else
f"{fmt_time(ev.time)}  {ev.title}"

                self.chips.addWidget(self._chip(label,
COLORS.get(ev.color, p['accent'])))

            if len(evs) > len(shown):

                more =
QLabel(f"+{len(evs)-len(shown)} more")


more.setStyleSheet(f"color:{p['text_dim']};
font-size:10px;")

                self.chips.addWidget(more)

        self.setToolTip("\n".join(e.title for e in evs)
if evs else (holiday or ""))



    def mousePressEvent(self, e):

        if self.date_key:
self.owner.open_event_dialog(self.date_key)

        super().mousePressEvent(e)




class ProCalendar(QWidget):

    def __init__(self):

        super().__init__()

        self.setWindowTitle("Pro Calendar 2026")

        self.setGeometry(100, 100, 1240, 820);
self.setMinimumSize(1080, 700)

        now = datetime.now()
```

```
        self.current_year, self.current_month =
now.year, now.month

        self.events = {}

        self.theme_mode = "light"

        self.load_events()

        self._build_ui()

        self._apply_theme()

        self.refresh()

        self.timer = QTimer(self);
self.timer.timeout.connect(self._tick);
self.timer.start(1000); self._tick()


    def _build_ui(self):

        root = QHBoxLayout(self);
root.setContentsMargins(0, 0, 0, 0);
root.setSpacing(0)

        root.addWidget(self._sidebar())

        root.addWidget(self._main_panel(), 1)


    def _sidebar(self):

        side = QFrame(); side.setFixedWidth(280)

        lay = QVBoxLayout(side);
lay.setContentsMargins(18, 22, 18, 18);
lay.setSpacing(14)


        self.create_btn = QPushButton("＋  Create
event"); self.create_btn.setFixedHeight(46)


self.create_btn.setCursor(QCursor(Qt.PointingHa
ndCursor))


self.create_btn.clicked.connect(self._create_toda
y)

        lay.addWidget(self.create_btn)


        nav = QHBoxLayout()
```

```
        self.mini_label = QLabel()

        prev, nxt = QToolButton(text="‹"),
QToolButton(text="›")

        prev.clicked.connect(self.prev_month);
nxt.clicked.connect(self.next_month)

        for b in (prev, nxt):
b.setCursor(QCursor(Qt.PointingHandCursor));
b.setFixedSize(26, 26)

        nav.addWidget(self.mini_label, 1);
nav.addWidget(prev); nav.addWidget(nxt)

        lay.addLayout(nav)



        self.mini_grid = QGridLayout();
self.mini_grid.setSpacing(4)

        lay.addLayout(self.mini_grid)

        self.mini_labels = []

        for col, d in enumerate(WD_MINI):

            l = QLabel(d);
l.setAlignment(Qt.AlignCenter)

            self.mini_grid.addWidget(l, 0, col)

        for r in range(1, 7):

            row = []

            for c in range(7):

                l = QLabel("");
l.setAlignment(Qt.AlignCenter);
l.setFixedSize(30, 26)


l.setCursor(QCursor(Qt.PointingHandCursor))

                l.mousePressEvent = lambda e, rr=r,
cc=c: self._mini_clicked(rr, cc)

                self.mini_grid.addWidget(l, r, c);
row.append(l)

            self.mini_labels.append(row)



        lay.addSpacing(10)

        lay.addWidget(QLabel("Upcoming"))
```

```
        scroll = QScrollArea();
scroll.setWidgetResizable(True);
scroll.setFrameShape(QFrame.NoFrame)

        scroll.setMaximumHeight(190)

        holder = QWidget(); self.upcoming_layout =
QVBoxLayout(holder)


self.upcoming_layout.setContentsMargins(0, 0,
0, 0); self.upcoming_layout.setSpacing(6)

        self.upcoming_layout.addStretch()

        scroll.setWidget(holder);
lay.addWidget(scroll)



        lay.addSpacing(6)

        sep1 = QFrame(); sep1.setFixedHeight(1);
sep1.setObjectName("sep")

        lay.addWidget(sep1)

        lay.addSpacing(6)



        lay.addWidget(QLabel("My Calendars"))

        legend = QGridLayout();
legend.setHorizontalSpacing(8);
legend.setVerticalSpacing(4)

        legend_names = list(COLORS.items())

        for i, (name, hexcode) in
enumerate(legend_names):

            dot = QLabel(); dot.setFixedSize(9, 9);
dot.setStyleSheet(f"background:{hexcode};
border-radius:4px;")

            lbl = QLabel(name);
lbl.setStyleSheet("font-size:11px;")

            r, c = divmod(i, 2)

            box = QHBoxLayout();
box.setSpacing(6); box.addWidget(dot);
box.addWidget(lbl); box.addStretch()

            wrap = QWidget(); wrap.setLayout(box)

            legend.addWidget(wrap, r, c)
```

```
        lay.addLayout(legend)


        lay.addSpacing(6)
        sep2 = QFrame(); sep2.setFixedHeight(1);
sep2.setObjectName("sep")
        lay.addWidget(sep2)
        lay.addSpacing(6)


        lay.addWidget(QLabel("This Month"))
        self.stats_label = QLabel("")
        self.stats_label.setWordWrap(True)
        lay.addWidget(self.stats_label)
        lay.addStretch()
        return side


    def _main_panel(self):
        panel = QWidget(); right =
QVBoxLayout(panel)
        right.setContentsMargins(0, 0, 0, 0);
right.setSpacing(0)


        self.header = QFrame();
self.header.setFixedHeight(190)
        self.bg_label = QLabel(self.header);
self.bg_label.lower()
        hl = QVBoxLayout(self.header);
hl.setContentsMargins(28, 16, 28, 16)
        top = QHBoxLayout()
        self.title_label = QLabel("Pro Calendar")
        self.clock_label = QLabel()
        self.settings_btn = QPushButton("⚙");
self.settings_btn.setFixedSize(40, 40)

self.settings_btn.setCursor(QCursor(Qt.Pointing
HandCursor))
```

```
self.settings_btn.clicked.connect(self._settings_m
enu)

        top.addWidget(self.title_label);
top.addStretch(); top.addWidget(self.clock_label)

        top.addSpacing(10);
top.addWidget(self.settings_btn)

        hl.addLayout(top); hl.addStretch()

        bottom = QHBoxLayout()

        self.month_year_label = QLabel();
bottom.addWidget(self.month_year_label);
bottom.addStretch()

        hl.addLayout(bottom)

        right.addWidget(self.header)



        body = QFrame(); bl =
QVBoxLayout(body)

        bl.setContentsMargins(24, 18, 24, 24);
bl.setSpacing(14)

        toolbar = QHBoxLayout()

        self.today_btn = QPushButton("Today");
self.today_btn.setFixedHeight(40)


self.today_btn.clicked.connect(self.go_to_today)

        prev, nxt = QToolButton(text="‹"),
QToolButton(text="›")

        prev.clicked.connect(self.prev_month);
nxt.clicked.connect(self.next_month)

        for b in (prev, nxt): b.setFixedSize(36, 36);
b.setCursor(QCursor(Qt.PointingHandCursor))

        self.toolbar_label = QLabel()

        toolbar.addWidget(self.today_btn);
toolbar.addWidget(prev); toolbar.addWidget(nxt)

        toolbar.addWidget(self.toolbar_label);
toolbar.addStretch()

        bl.addLayout(toolbar)
```

```
        self.grid_widget = QWidget(); self.grid =
QGridLayout(self.grid_widget);
self.grid.setSpacing(1)

        for c in range(7):
self.grid.setColumnStretch(c, 1)

        for r in range(1, 7):
self.grid.setRowStretch(r, 1)

        for col, day in enumerate(WD):

            l = QLabel(day);
l.setAlignment(Qt.AlignCenter)

            self.grid.addWidget(l, 0, col)

        self.day_cells = []

        for r in range(1, 7):

            row = []

            for c in range(7):

                cell = DayCell(self);
self.grid.addWidget(cell, r, c); row.append(cell)

            self.day_cells.append(row)

        bl.addWidget(self.grid_widget, 1)

        right.addWidget(body, 1)

        return panel


    def refresh(self):

        p = self._palette


self.month_year_label.setText(f"{calendar.month
_name[self.current_month]}
{self.current_year}")


self.toolbar_label.setText(f"{calendar.month_na
me[self.current_month]}  {self.current_year}")

        self._fill_main_grid(p)

        self._fill_mini_grid()

        self._fill_upcoming()

        self._load_background(self.current_month)
```

```
    def _fill_main_grid(self, p):
        days =
list(calendar.Calendar(firstweekday=0).itermonth
days(self.current_year, self.current_month))
        today = datetime.now(); hd =
holidays(self.current_year); it = iter(days)
        for week in range(6):
            for col in range(7):
                cell = self.day_cells[week][col]
                d = next(it, 0)
                if d == 0:
                    cell.set_empty(p); continue
                key =
f"{self.current_year}-{self.current_month:02d}-{
d:02d}"
                evs = self.events.get(key, [])
                is_today = today.year ==
self.current_year and today.month ==
self.current_month and today.day == d
                cell.set_day(d, evs, hd.get(key),
is_today, col >= 5, key, p)


    def _fill_mini_grid(self):

self.mini_label.setText(f"{calendar.month_name[
self.current_month]} {self.current_year}")
        days =
list(calendar.Calendar(firstweekday=0).itermonth
days(self.current_year, self.current_month))
        today = datetime.now(); it = iter(days)
        for week in range(6):
            for col in range(7):
                l = self.mini_labels[week][col]
                d = next(it, 0)
                if d == 0:
                    l.setText(""); l.setStyleSheet("");
continue
```

```
            key =
f"{self.current_year}-{self.current_month:02d}-{
d:02d}"

            is_today = today.year ==
self.current_year and today.month ==
self.current_month and today.day == d

            l.setText(str(d))

            p = self._palette

            style = f"color:{p['text']};
border-radius:13px;"

            if is_today: style =
f"background:{p['accent']}; color:white;
font-weight:700; border-radius:13px;"

            elif self.events.get(key): style += "
font-weight:700; text-decoration:underline;"

            l.setStyleSheet(style)


    def _fill_upcoming(self):

        while self.upcoming_layout.count():

            w =
self.upcoming_layout.takeAt(0).widget()

            if w: w.deleteLater()

        today = datetime.now().date(); flat = []

        for key, evs in self.events.items():

            try: d = datetime.strptime(key,
"%Y-%m-%d").date()

            except ValueError: continue

            if d >= today:

                for ev in evs: flat.append((d, ev))

        flat.sort(key=lambda x: (x[0], x[1].time or
"99:99"))

        flat = flat[:8]

        if not flat:

            l = QLabel("No upcoming events");
l.setStyleSheet(f"color:{self._palette['text_dim']}
; font-size:12px;")

            self.upcoming_layout.addWidget(l)
```

```
        else:
            for d, ev in flat:
                row = QFrame(); h = QHBoxLayout(row); h.setContentsMargins(8, 6, 8, 6); h.setSpacing(8)
                dot = QLabel(); dot.setFixedSize(10, 10)
dot.setStyleSheet(f"background:{COLORS.get(ev.color,'#1a73e8')}; border-radius:5px;")
                txt = QLabel(f"{ev.title}\n{d.strftime('%b %d')}" + (f" · {fmt_time(ev.time)}" if ev.time else ""))
                h.addWidget(dot); h.addWidget(txt, 1)
row.setStyleSheet(f"background:{self._palette['bg_alt']}; border-radius:8px;")
                self.upcoming_layout.addWidget(row)
        self.upcoming_layout.addStretch()

    def open_event_dialog(self, date_key):
        existing_list = self.events.get(date_key, [])
        existing = existing_list[0] if existing_list else None
        dlg = EventDialog(self, date_key, existing)
        if dlg.exec() == QDialog.Accepted:
            if dlg.delete_requested:
                if existing_list:
                    existing_list.pop(0)
                    if not existing_list: self.events.pop(date_key, None)
            elif dlg.result_event:
                lst = self.events.setdefault(date_key, [])
                if existing: lst[0] = dlg.result_event
                else: lst.append(dlg.result_event)
            self.save_events(); self.refresh()
```

```
    def _create_today(self):
        now = datetime.now()
        if now.month != self.current_month or
now.year != self.current_year:
            self.current_year, self.current_month =
now.year, now.month
            self.refresh()

self.open_event_dialog(now.strftime("%Y-%m-%
d"))

    def _mini_clicked(self, row, col):
        l = self.mini_labels[row][col]
        if l.text().isdigit():
            day = int(l.text())

self.open_event_dialog(f"{self.current_year}-{se
lf.current_month:02d}-{day:02d}")

    def _settings_menu(self):
        menu = QMenu(self)
        for label, mode in (("☀  Light theme",
"light"), ("🌙  Dark theme", "dark"), ("🖥️
Match system", "system")):
            act = QAction(label, self);
act.setCheckable(True);
act.setChecked(self.theme_mode == mode)
            act.triggered.connect(lambda _, m=mode:
self._set_theme(m))
            menu.addAction(act)

menu.exec(self.settings_btn.mapToGlobal(self.se
ttings_btn.rect().bottomLeft()))

    def _set_theme(self, mode):
        self.theme_mode = mode
```

```
        self._apply_theme()
        self.refresh()

    def _apply_theme(self):
        mode = system_theme() if self.theme_mode
== "system" else self.theme_mode
        p = PALETTES[mode]; self._palette = p
        self.setStyleSheet(f"""
            QWidget {{ font-family:'Segoe
UI',Arial,sans-serif; color:{p['text']}; }}
            ProCalendar {{ background:{p['bg']}; }}
            QPushButton {{
background:{p['bg_alt']}; border:1px solid
{p['border']}; border-radius:18px; padding:0
16px; }}
            QPushButton:hover {{
background:{p['hover']}; }}
            QToolButton {{ background:transparent;
border:none; font-size:16px; border-radius:13px;
}}
            QToolButton:hover {{
background:{p['hover']}; }}
            QLabel {{ color:{p['text']}; }}
            DayCell {{ background:{p['bg']};
border:1px solid {p['border']}; }}
            DayCell:hover {{
background:{p['hover']}; }}
        """)

self.title_label.setStyleSheet("font-size:20px;
font-weight:600; color:white;")

self.clock_label.setStyleSheet("font-size:16px;
font-family:Consolas; color:white;")

self.month_year_label.setStyleSheet("font-size:3
0px; font-weight:700; color:white;")

self.settings_btn.setStyleSheet("background:rgba
```

```
(255,255,255,40); border:none;
border-radius:20px; color:white;")


self.create_btn.setStyleSheet(f"background:{p['ac
cent']}; color:white; border:none;
border-radius:23px; font-weight:600;
text-align:left; padding-left:18px;")


self.toolbar_label.setStyleSheet("font-size:20px;
font-weight:600; margin-left:8px;")

        self._load_background(self.current_month)


    def _tick(self):


self.clock_label.setText(datetime.now().strftime("
%I:%M:%S %p").lstrip("0"))



    def prev_month(self):

        self.current_month -= 1

        if self.current_month < 1:
self.current_month, self.current_year = 12,
self.current_year - 1

        self.refresh()



    def next_month(self):

        self.current_month += 1

        if self.current_month > 12:
self.current_month, self.current_year = 1,
self.current_year + 1

        self.refresh()



    def go_to_today(self):

        now = datetime.now(); self.current_year,
self.current_month = now.year, now.month

        self.refresh()



    def load_events(self):
```

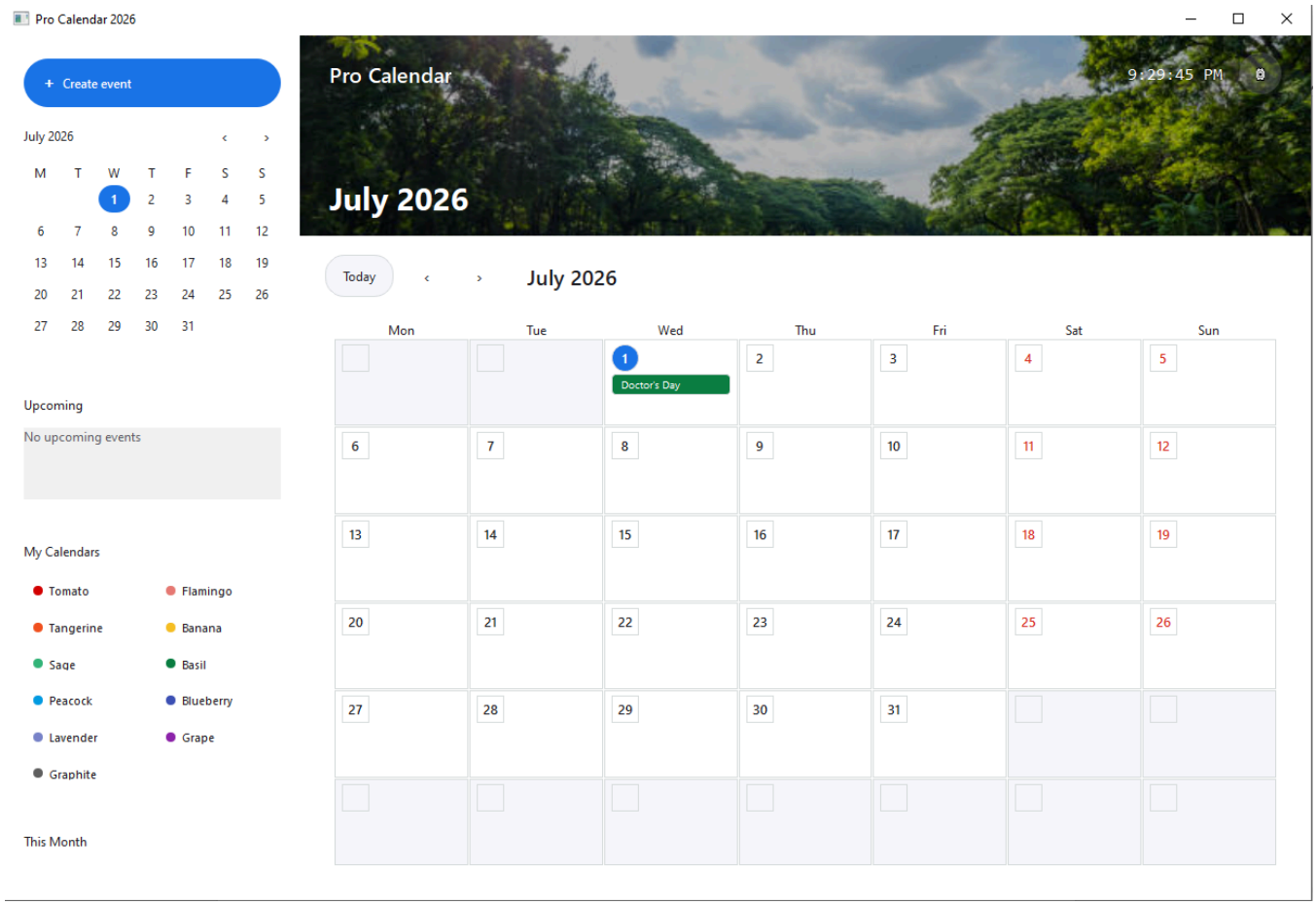
```
        self.events = {}
        if not os.path.exists(DATA_FILE): return
        try:
            with open(DATA_FILE, "r",
encoding="utf-8") as f:
                raw = json.load(f)
            for key, val in raw.items():
                if isinstance(val, list): self.events[key]
= [Event.from_dict(v) for v in val]
                elif isinstance(val, dict):
self.events[key] = [Event.from_dict(val)]
                elif isinstance(val, str): self.events[key]
= [Event(val)]
        except (json.JSONDecodeError, OSError):
            self.events = {}


    def save_events(self):
        try:
            data = {k: [e.to_dict() for e in v] for k, v
in self.events.items() if v}
            with open(DATA_FILE, "w",
encoding="utf-8") as f:
                json.dump(data, f, indent=2)
        except OSError as exc:
            QMessageBox.warning(self, "Save
failed", f"Could not save events:\n{exc}")


    def _load_background(self, month):
        w, h = max(self.header.width(), 900),
self.header.height()
        pix = None
        try:
            path = os.path.join(IMG_DIR,
IMG_NAMES[month - 1])
            if os.path.isfile(path) and
os.path.getsize(path) > 0:
```

```
                cand = QPixmap(path)
                if not cand.isNull(): pix = cand
            except Exception:
                pix = None
            if pix is None:
                pix = self._gradient(w, h, month)
            else:
                pix = pix.scaled(w, h,
Qt.KeepAspectRatioByExpanding,
Qt.SmoothTransformation)
                if pix.width() > w or pix.height() > h:
                    x, y = max(0, (pix.width()-w)//2),
max(0, (pix.height()-h)//2)
                    pix = pix.copy(x, y, w, h)
                d = QPixmap(pix); pt = QPainter(d);
pt.fillRect(d.rect(), QColor(0, 0, 0, 70)); pt.end()
                pix = d
            self.bg_label.setPixmap(pix);
self.bg_label.setGeometry(0, 0, w, h);
self.bg_label.lower()


    @staticmethod
    def _gradient(w, h, month):
        c1, c2 = GRADIENTS[month - 1]
        pix = QPixmap(max(w, 1), max(h, 1)); pt =
QPainter(pix)
        g = QLinearGradient(0, 0, w, h);
g.setColorAt(0, QColor(c1)); g.setColorAt(1,
QColor(c2))
        pt.fillRect(pix.rect(), g); pt.end()
        return pix


    def resizeEvent(self, e):
        super().resizeEvent(e)
        self._load_background(self.current_month)
```

```
# ──────────────────────────────────────────────────────────────────────────
#  ENTRY POINT
# ──────────────────────────────────────────────────────────────────────────

def main():
    app = QApplication(sys.argv)
    app.setStyle("Fusion")

    cal_win = None

    def open_calendar():
        nonlocal cal_win
        cal_win = ProCalendar()
        # Fade the calendar in
        effect = QGraphicsOpacityEffect(cal_win)
        cal_win.setGraphicsEffect(effect)
        effect.setOpacity(0)
        cal_win.show()
        anim = QPropertyAnimation(effect,
b"opacity")
        anim.setStartValue(0.0)
        anim.setEndValue(1.0)
        anim.setDuration(500)

anim.setEasingCurve(QEasingCurve.OutCubic)
        anim.start()
        # keep reference so it isn't GC'd
```

```
        cal_win._fade_anim = anim


    splash = SplashScreen(open_calendar)
    # Center on screen
    screen = app.primaryScreen().geometry()
    splash.move((screen.width() - splash.width())
// 2,
              (screen.height() - splash.height()) // 2)
    splash.show()


    sys.exit(app.exec())



if __name__ == "__main__":
    main()
```

Result:



Conclusions:

This lab successfully demonstrated the shift from terminal-based programs to interactive visual applications. By designing layouts in Qt Designer and linking their components to backend Python logic via signals and slots, a solid understanding of event-driven software development and professional user interface design was achieved..

University Of Engineering and Technology, Lahore **Computer Engineering Department**

Course Name: Programming Fundamentals	**Course Code:** CMPE-112L
Assignment Type: Manual Report	**Dated: 28/06/2026**
Semester: 1st	**Session: 2026-30**
Lab/Project/Assignment #: 1(sir Mujtaba)	**CLOs to be covered: CLO 2**
Lab Title: Basics/Intro	**Teacher Name:** Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	**Marks**	**Level**	**Rubric**
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.

Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 16

CLO 1:

Lab Goals/Objectives:

UET Test portal

Equipment Required:

- Hardware: Laptop / PC
- Software: IDE (e.g., PyCharm / VS Code), Python/C Environment.

Lab Work:

Task 1:

```
import time
import random

# ── Configuration ──────────────────────────────────────────────
RIGHT_POINTS = 4
WRONG_POINTS = -1
NO_ATTEMPT_POINTS = 0
LOGIN_TRIES = 3

# ── Login Details ──────────────────────────────────────────────
ADMIN_ID = "Admin"
ADMIN_PW = "12345"
STD_ID = "Dawar"
STD_PW = "12345"

# ── Updated Physics-Focused Question Bank ──────────────────────
quiz_items = [
{"id":1, "topic":"Physics", "text":"What is the SI unit of pressure?", "opts":{"A":"Pascal", "B":"Joule", "C":"Watt", "D":"Newton"}, "correct":"A"},
{"id":2, "topic":"Physics", "text":"The formula for potential energy is?", "opts":{"A":"1/2 mv²", "B":"mgh", "C":"mv", "D":"F×d"}, "correct":"B"},
{"id":3, "topic":"Math", "text":"Cos(0°) equals?", "opts":{"A":"1", "B":"0", "C":"0.5", "D":"-1"}, "correct":"A"},
{"id":4, "topic":"Physics", "text":"Which law explains why we wear seatbelts?", "opts":{"A":"Newton's 1st Law", "B":"Newton's 2nd Law", "C":"Newton's 3rd Law", "D":"Law of Gravitation"}, "correct":"A"},
```

{"id":5, "topic":"Chemistry", "text":"Atomic number of Oxygen?", "opts":{"A":"6", "B":"7", "C":"8", "D":"9"}, "correct":"C"},
{"id":6, "topic":"Physics", "text":"1 kWh is equal to how many Joules?", "opts":{"A":"3.6×10^{6}", "B":"3.6×10^{5}", "C":"3.6×10^{7}", "D":"3.6×10^{8}"}, "correct":"A"},
{"id":7, "topic":"Math", "text":"Square root of 169?", "opts":{"A":"11", "B":"12", "C":"13", "D":"14"}, "correct":"C"},
{"id":8, "topic":"Physics", "text":"Reflection of light follows which law?", "opts":{"A":"Angle i = Angle r", "B":"i + r = 90°", "C":"i > r", "D":"i < r"}, "correct":"A"},
{"id":9, "topic":"Physics", "text":"Unit of frequency?", "opts":{"A":"Hertz", "B":"Newton", "C":"Pascal", "D":"Watt"}, "correct":"A"},
{"id":10, "topic":"Chemistry", "text":"Symbol of Potassium?", "opts":{"A":"P", "B":"Po", "C":"K", "D":"Pt"}, "correct":"C"},
{"id":11, "topic":"Physics", "text":"Current is the rate of flow of?", "opts":{"A":"Charge", "B":"Voltage", "C":"Resistance", "D":"Power"}, "correct":"A"},
{"id":12, "topic":"Physics", "text":"What is the escape velocity on Earth (approx)?", "opts":{"A":"7.8 km/s", "B":"11.2 km/s", "C":"9.8 km/s", "D":"3×10^{8} m/s"}, "correct":"B"},
]

student_records = []

# ── Display Helpers ──
def separator():
print("=" * 55)

def show_title(title):
separator()
print(f" {title.center(45)}")
separator()

def performance_grade(percentage):
if percentage >= 80: return "EXCELLENT"
if percentage >= 65: return "GOOD"
if percentage >= 50: return "AVERAGE"
return "NEEDS IMPROVEMENT"

def compute_result(user_responses):
right = wrong = skipped = 0
for index, item in enumerate(quiz_items):
reply = user_responses.get(index, "S")
if reply == "S":
skipped += 1
elif reply == item["correct"]:
right += 1
else:
wrong += 1
obtained = right * RIGHT_POINTS + wrong * WRONG_POINTS
total_possible = len(quiz_items) * RIGHT_POINTS
percent = round((obtained / total_possible) * 100, 2) if total_possible else 0
return right, wrong, skipped, obtained, total_possible, percent

def authenticate(portal_type, valid_id, valid_pw):
show_title(f"{portal_type} Login")
for trial in range(1, LOGIN_TRIES + 1):

```
uid = input("Enter Username : ").strip()
pwd = input("Enter Password : ").strip()
if uid == valid_id and pwd == valid_pw:
print("\n✅ Login Successful!\n")
time.sleep(0.7)
return True
print(f"❌ Wrong credentials. Attempt {trial}/{LOGIN_TRIES}")
print("⛔ Login failed. Try again later.\n")
return False

# ── Admin Features ──────────────────────────────────────────────
def show_all_questions():
show_title("Question Bank")
for num, item in enumerate(quiz_items, 1):
print(f"\nQ{num}. [{item['topic']}] {item['text']}")
for let, txt in item["opts"].items():
print(f" {let}) {txt}")
print(f" Correct: {item['correct']}")

def add_new_item():
show_title("Add New Question")
sub = input("Subject: ").strip()
que = input("Question Text: ").strip()
opts = {}
for ch in ["A","B","C","D"]:
opts[ch] = input(f"Option {ch}: ").strip()
ans = input("Correct Option (A/B/C/D): ").strip().upper()
if ans not in "ABCD":
print("Invalid option!")
return
quiz_items.append({"id": len(quiz_items)+1, "topic": sub, "text": que, "opts": opts, "correct": ans})
print("✅ Question added successfully!")

def remove_item():
show_all_questions()
try:
idx = int(input("\nEnter question number to remove: ")) - 1
if 0 <= idx < len(quiz_items):
quiz_items.pop(idx)
print("✅ Question removed!")
else:
print("Invalid number.")
except:
print("Please enter a valid number.")

def bank_overview():
show_title("Bank Overview")
print(f"Total Questions : {len(quiz_items)}")
count_map = {}
for q in quiz_items:
count_map[q["topic"]] = count_map.get(q["topic"], 0) + 1
for sub, cnt in count_map.items():
```

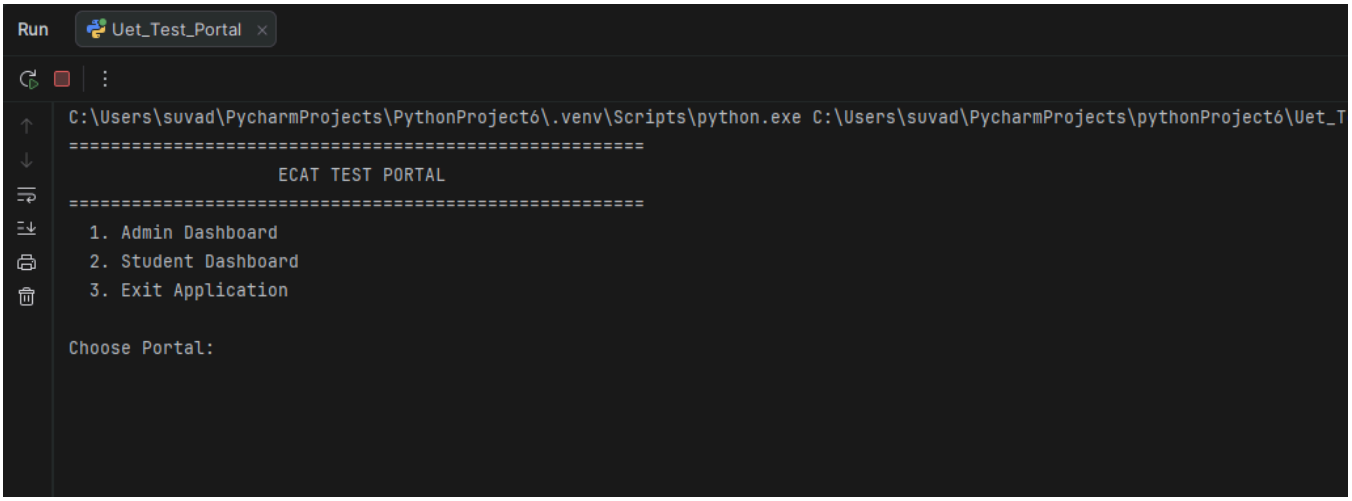
```
print(f" {sub}: {cnt}")

def show_records():
show_title("All Student Records")
if not student_records:
print("No records available yet.")
return
print(f"{'No':<3} {'Name':<15} {'Roll':<10} {'Marks':<7} {'%':<6} {'Grade':<15} {'Date'}")
separator()
for i, rec in enumerate(student_records, 1):
print(f"{i:<3} {rec['name']:<15} {rec['roll']:<10} {rec['marks']:<7} {rec['perc']:<6} {rec['grade']:<15}
{rec['timestamp']}")

def detailed_view():
show_records()
if not student_records: return
try:
pos = int(input("\nEnter record number: ")) - 1
rec = student_records[pos]
except:
print("Invalid choice.")
return
show_title(f"Review - {rec['name']}")
for i, item in enumerate(quiz_items):
ans_given = rec["responses"].get(i, "S")
mark = "✅" if ans_given == item["correct"] else ("—" if ans_given == "S" else "❌")
print(f"Q{i+1}. {item['text']}")
print(f" Answered: {ans_given} Correct: {item['correct']} {mark}")

def overall_stats():
show_title("Class Performance")
if not student_records:
print("No data yet.")
return
marks_list = [r["marks"] for r in student_records]
perc_list = [r["perc"] for r in student_records]
passed_count = sum(1 for p in perc_list if p >= 50)
print(f"Highest Marks : {max(marks_list)}")
print(f"Lowest Marks : {min(marks_list)}")
print(f"Average Marks : {sum(marks_list)/len(marks_list):.1f}")
print(f"Pass Percentage : {passed_count}/{len(student_records)}")
for g in ["EXCELLENT","GOOD","AVERAGE","NEEDS IMPROVEMENT"]:
print(f" {g}: {sum(1 for r in student_records if r['grade']==g)}")

def admin_section():
if not authenticate("Admin", ADMIN_ID, ADMIN_PW): return
options = {
"1": ("Show All Questions", show_all_questions),
"2": ("Add Question", add_new_item),
"3": ("Remove Question", remove_item),
"4": ("Bank Overview", bank_overview),
"5": ("All Records", show_records),
```

```
"6": ("Detailed Review", detailed_view),
"7": ("Class Statistics", overall_stats),
"8": ("Logout", None)
}
while True:
show_title("Admin Dashboard")
for num, (label, _) in options.items():
print(f" {num}. {label}")
sel = input("\nEnter choice: ").strip()
if sel == "8": break
if sel in options and options[sel][1]:
options[sel][1]()
else:
print("Invalid choice.")

# ── Student Section ──────────────────────────────────────────────
def display_rules():
show_title("Examination Guidelines")
print("• Choose A, B, C or D")
print("• Type S to skip a question")
print(f"• +{RIGHT_POINTS} for correct | {WRONG_POINTS} for wrong |
{NO_ATTEMPT_POINTS} for skip")
print("• You can type SUBMIT to finish early")

def begin_test(stu_name, stu_roll):
show_title("ECAT Test in Progress")
responses = {}
for idx, item in enumerate(quiz_items):
print(f"\nQ{idx+1}/{len(quiz_items)}. [{item['topic']}] {item['text']}")
for k, v in item["opts"].items():
print(f" {k}) {v}")
while True:
reply = input("\nYour Answer (A/B/C/D/S | SUBMIT): ").strip().upper()
if reply == "SUBMIT":
print("Test submitted early.")
break
if reply in ["A","B","C","D","S"]:
responses[idx] = reply
break
print("Invalid input. Please try again.")
if reply == "SUBMIT": break

right, wrong, skip, marks, total, perc = compute_result(responses)
grd = performance_grade(perc)
current_time = time.strftime("%d-%b-%Y %H:%M")

student_records.append({
"name": stu_name,
"roll": stu_roll,
"marks": marks,
"perc": perc,
"grade": grd,
```

```
"timestamp": current_time,
"responses": responses,
"right": right,
"wrong": wrong,
"skip": skip
})

show_title("Test Result")
print(f"Student : {stu_name}")
print(f"Roll No. : {stu_roll}")
print(f"Score : {marks} / {total}")
print(f"Percentage : {perc}%")
print(f"Grade : {grd}")
print(f"\nCorrect : {right} | Wrong : {wrong} | Skipped : {skip}")
separator()
print("\nQuestion-wise Review:")
for i, item in enumerate(quiz_items):
given = responses.get(i, "S")
res = "✅" if given == item["correct"] else ("—" if given == "S" else "❌")
print(f"Q{i+1}. {res} Your Ans: {given} Correct: {item['correct']}")

def student_section():
if not authenticate("Student", STD_ID, STD_PW): return
show_title("Student Dashboard")
name = input("Enter Full Name : ").strip()
roll = input("Enter Roll No : ").strip()
options = {"1": "Start Test", "2": "View Rules", "3": "Logout"}
while True:
show_title("Student Menu")
for k, v in options.items():
print(f" {k}. {v}")
choice = input("\nSelect option: ").strip()
if choice == "1":
begin_test(name, roll)
elif choice == "2":
display_rules()
elif choice == "3":
break
else:
print("Invalid option.")

# ── Main Entry Point ──────────────────────────────────────────────
def main():
while True:
show_title("ECAT TEST PORTAL")
print(" 1. Admin Dashboard")
print(" 2. Student Dashboard")
print(" 3. Exit Application")
sel = input("\nChoose Portal: ").strip()
if sel == "1":
admin_section()
elif sel == "2":
```

```
student_section()
elif sel == "3":
print("\nThank you for using ECAT Portal. Goodbye!")
break
else:
print("Please select a valid option.")

if __name__ == "__main__":
main()
```

Result:



Conclusions:

This lab session successfully consolidated the practical mechanics of functions in software design. By transitioning from basic greeting scripts to developing calculations for arithmetic squares, exponents, maximum finders, averages, and dynamic lambda-driven multiplication tables, a robust control over function returns and modular execution was established. All code blocks executed cleanly with accurate outputs..